Performance Analysis and Assessment for Computational Research Software—A Workbook

Tobias Weinzierl and Thomas Flynn

May 15, 2026

Preface

April 2025

We organised a series of performance analysis workshops in spring 2025, funded through the Digital Research Infrastructure (DRI) programme under the umbrella of our HAI-End project. These workshops differed from previous installments in that we did not focus on particular tools. Instead, we conducted brainstorming and discussion exercises centered around a fundamental question: How do you begin performance assessment when you know little to nothing about your code?

While some tools such as Intel's Application Performance Snapshots (APS) or Linaro's MAP provide high-level overviews, we approached this challenge not from a tools perspective but from an observational point of view. We asked: What are the essential properties we must understand before diving deeper into code analysis?

This inquiry led to a high-level, top-down approach, documented through mindmaps, flowcharts, and case studies. The raising approach is fundamentally different from the existing POP methodology, which collects fine-grained metrics and successively aggregates them into high-level metrics and flaws. It also differs from approaches offered by and supported through tools like Scalasca, which require a certain level of code maturity and system knowledge from the user.

Both direction of travel are important and very valuable, and we think they might be more sophisticated and insightful compared to what we came up with. However, our technique is very simple and does require next to no specialised software and tools knowledge. We developed a straightforward, accessible starting point to initiating performance analysis. This first version of the document summarises these foundational ideas.

Some of the sections are not yet written, and a lot more are yet to come. However, we decided to share them early, to be able to gather feedback and polish the ideas. This is only the first version of a living document.

Take home: - Target group has to be clearer: in-depth or application-domain focused - Tool information density plus training material is not appropriate for latter group - Future workshops have to cover both domains separately

- Application people want to do performance engineering but don't know how to do (and that they have to do) assessment first - They pick the first tool and then stick to it and hence often miss out on big picture

- Success model: pair up tool expert with domain expert - Tool training is there for

assessor, but not necessarily for the domain expert who wants to do performance engineering - This contradicts my previous assumption that tools are there predominantly for performance engineer

- Maybe the assessments have to come first - Success stories are key (like the ones POP has delivered)

- A lot of tools are too difficult to use; counterpart is MAQAO or APS or Perf Snapshot

- Pitch assessment gurus directly for tool workshops - Create for others a tutorial ”how to assess your code’s performance”

Thomas Flynn, Tobias Weinzierl
Durham, May 15, 2026

Contents

I Introduction **9**

1 How to read **11**

2 Terminology **15**
2.1 What we do 15
2.2 Code development workflows 16
2.3 Fundamental HPC and assessment terms 18
2.4 Bottom-up vs. top-down assessment 21
2.5 Symptomatic vs. causal assessment 21

II High Level Overview **23**

3 Process **25**
3.1 Goals 26
3.2 Criticism 27
3.3 Shortcomings 27

4 Preparation **29**
4.1 Benchmark characteristics 29
4.2 Working environment 32
4.3 Compiler setup 34
4.4 Code scaling and complexity 35
4.5 Memory, storage and I/O 36
4.6 Wrap-up 37

5 High-level first glance **39**
5.1 The assessment rubrics 40
5.2 Core performance 41
5.3 GPU performance 44
5.4 I/O performance 46
5.5 Intra-node (node-level) performance 48
5.6 Inter-node performance 53
5.7 Wrap-up 57

III Deep Dive 59

6 Workflow 61
6.1 Localisation . . . 61
6.2 Navigation . . . 64
6.3 Affinity . . . 66
6.4 Process placement . . . 66

7 Core 67
7.1 Compiler optimisations . . . 67

8 Node-level (intra-node) analysis 69
8.1 Coverage . . . 70
8.2 Balancing . . . 74
8.3 Concurrency level vs. observed parallelisam . . . 76
8.4 Tasks are too small . . . 77
8.5 Loop segments are too small . . . 79
8.6 Tasks are too big . . . 79
8.7 NUMA effects . . . 79
8.8 Synchronisation . . . 79
8.9 Data dependencies . . . 79
8.10 Redundant (administrative) work . . . 79
8.11 Homogeneous task duration . . . 79
8.12 Hierarchical decomposition . . . 79
8.13 Excessively fine-granular task synchronisation . . . 79

IV Resources 81

9 Tools 83
9.1 GNU profiler (gprof) . . . 83
9.2 Python's on-board profiling . . . 84
9.3 APS . . . 84
9.4 Intel VTune . . . 84
9.5 Score-P . . . 85
9.6 Linaro Map . . . 85
9.7 MAQAO . . . 85
9.8 Intel Amplifier . . . 85

V Case Studies 87

10 High-level assessments 89

VI Appendix **91**

A Acknowledgements **93**
A.1 Funding councils and supporting initiatives 93
A.2 People 93
A.3 Partners 94

B Submission checklist **95**
B.1 Code submission 95
B.2 Code of conduct 98
B.3 Outcomes 98

C Choice of Metrics **99**
C.1 GPU Metric: Occupancy or FLOPS? 99

D Communication with code owner **101**
D.1 Existing optimisations 101

Part I

Introduction

Chapter 1

How to read

As with any workbook — i.e. a book that's there to guide actual work not to teach a big subject — there are numerous ways to read the text. You can read the manuscript chapter by chapter, but this is unlikely to be the most fun experience. Below, we collect some ideas on how to substructure the reading:

Context

This document is part of an ongoing body work of designing and implementing a structured performance analysis methodology with the future aim of building a performance assessment service.

The work presented here is a precursor to a far more detailed text which will include a structured methodology for conducting performance assessments. In this iteration we introduce: relevant terminology for High-Performance Computing (HPC) and performance analysis; preparing for an assessment; and finally how to begin an assessment from a high-level.

Current state of manuscript

Both Chapter 2 and 4 are predominantly complete though with notable exceptions such as Section 4.4 which are yet to be completed, and are labelled as 't.b.d'.

Within Chapter 5, the first three sections are largely finished and have been written in conjunction with implementing these methodologies during recent performance analysis workshops. Section 5.6 has been started but the ending of this section still needs significant work. We want to implement more of this 'inter-node' assessment before making recommendations that are unfounded. Sections 5.3 and 5.4 are yet to be worked through, though Section **??** is mostly complete.

Clearly sections labelled as 'complete' may also need to be extended as further work is done, as the content may become insufficient. Note however that some forward references may link to topics to come in future versions of this manuscript, e.g. discussions of tools, performance assessment service, etc.

Finally, Chapter **??** will not be part of the final document (as with this section and the previous) but recaps the current progress of this document whilst also discussing the content to come in the next iteration of this document.

First-time assessment

We recommend to run through the preparatory remarks in Chapter 4 first and to complete all the checkboxes (dos) there. Once completed, it is time to run through the high-level assessment remarks in Chapter 5. With most codes, it is highly unlikely that a code will only show one kind of flaw in the high-level analysis.

We propose to run through the detailed analysis step by step afterwards. Per step, it makes sense to consult the tools summary (Chapter 9) and notably to try out different things. Where appropriate, the reader might prefer to read through one of the case studies in Part **??**, just to get some inspiration on what tools and approaches to try out and how to report outcomes.

Assessment

If you have already completed numerous assessments, then the manuscript is mainly a reference document. You'd start with the high-level assessment in Chapter 5, before you dig into the various subchapters depending on the initial assessment outcome. The case studies are not of particular relevance (anymore), as you have already completed you own case studies.

Feedback

The quintessential objective of any assessment is to give feedback to developers. Once they make changes to their source code, it is important to re-start the assessment from scratch. There is always this temptation to return to the particular characteristics of interest that have led to a report, to see how the code changes by the development team have altered these characteristics, and then to continue from there.

It is important to reassess the "flaw" metrics. However, this should only be a validation that something has changed. After that, we have to start from scratch, as performance optimisation is a complicated process where one change might introduce flaws, improvement or character changes in a completely different part of the code or affect a completely different flavour of the performance. If we alter a particular code part that makes the scaling deteriorate for example, this might have a severe knock on effect on the core performance. If we improve the core performance, the GPU offloading might all of a sudden run into data transfer issues. There are numerous examples for these side-effects.

How it works

- ⊞ Conduct a performance assessment which both covers all aspects of the code (cmp. Chapter 5) plus goes in-depth all the way through for all flaws identified.
- ⊞ Hand this completed assessment back to the developers to address the flaws.
- ⊞ Restart the assessment from scratch top-down, i.e. starting with the high-level assessment covering all code runtime nuances.

How it does not work

- ⊟ Mix code optimisation and performance assessment workflows.
- ⊟ Restart the assessment from the place where you stopped when you reported a flaw and continue digging through the assessment decision metrics without redoing a high-level, overarching assessment.

Chapter 2

Terminology

Performance and efficiency, i.e. the speed of a code, are critical non-functional properties of scientific codes. Measuring, assessing and interpreting them is challenging; without even considering tuning. Most annoyingly, there is no standardised way how to analyse and assess performance. There is no vanilla workflow. Every team, every project, every scientist create their own analysis workflow or process, and this processes often changes from paper to paper or experiment to experiment.

Despite the vagueness in the metrics and the processes, there are some well-defined, common terms in the community that we should use. There are also some key concepts and some jargon that we use throughout the write-up. So we better introduce them first.

2.1 What we do

From hereon, we rely on a few key assumptions:

1. We assume that the person studying the code has *limited knowledge* of the code's internals, the application domain, and the algorithms used.

2. We evaluate *how* a code performs and whether certain characteristics deserve closer examination.

3. We are interested in the code's behavior on *one particular machine* for *one particular experimental setup* or a closely defined set of experiments.

4. Whenever we explain *why* a code performs as it does, this explanation comes from a machine's point of view: It does not argue about the algorithm or science case.

Doctrine 1 Performance Assessment *documents code behaviour from an outsider's point of view. It captures what we see (benchmarking).* Performance Analysis *explains why a code behaves the way it does.* Performance Engineering *improves the performance.*

The three steps typically build up on each other. Performance assessment is a little bit like a health screening in medicine. We run a couple of standard benchmarks (similar to blood pressure, oxygen saturation and pulse for example). If some metrics are not in the region where we want them to be, we issue further tests, i.e. we switch to the analysis part.

The skills we need first and foremost are to know what tests we should conduct or commission, and we have to be able to read the outcomes. If the test metrics are not what they should be, we eventually transfer the patient (code) to a specialist and run further tests. Based upon these tests, we finally start the treatment.

Reader's guide

This text starts with a performance assessment before it switches to some performance analysis. It does not cover aspects of performance engineering.

Our working assumption is that performance engineering requires inevitably domain or code experise which we cannot/might not have as generalists. However, we also assume that there is value in and a need for indepenent, unbiased analysis before someone dives into a code. More rationale are subject of discussion Chapter 3.

Assessment crime 1 *Developers tend to skip the assessment part (sometimes even the analysis) and dive straight into performance tweaking.*

They want to get their hands dirty and improve the code. However, this is deeply unscientific and unreasonable without a proper assessment and triggers lots of follow-up complications. Systematic work on performance always follows the sequence assessment–analysis–improvement (engineering).

How it works

- ⊞ Ignore everything you know nothing about the code and focus on the data you measure.
- ⊞ Stop any development while you assess and analyse.

How it does not work

- ⊟ Start with the attitude "I want to fix a flaw".
- ⊟ Skip assessment steps, as you already know what's going on.

2.2 Code development workflows

Reader's guide

Feel free to skip the remainder of this chapter if you are a first-time reader.

Systematic scientific computing typically implements an iterative approach: First, we measure or benchmark how a code performs and use these data to inform our performance assessment, i.e. our reporting. Second, we conduct performance analysis to explain why the code exhibits the observed behaviour. Finally, this analysis feeds into program tuning or optimisation. After that, we repeat the workflow.

If we take our list of activities and integrate it within the definition of the performance analysis workflow, it becomes clear that the tuning step is out-of-scope here. Another related technique is out-of-scope, too: The most advanced papers in scientific computing, particularly within high-performance computing, deliver a performance model; essentially a pen-and-paper assessment of what a code should be able to deliver. This is impossible if we adhere to the principles above: having limited knowledge and only explaining observed behavior rather than relating it to external factors (such as input data or algorithmic choices) or algorithmic context. We commit to a data-driven approach. Obviously, such an approach might complement an analytic, theoretical assessment of code, but we recommend considering this as an entirely separate cup of tea.

We will inevitably face difficulties if we truly do not know the code we are evaluating: Explaining certain effects is so much easier with some (rudimentary) understanding of the code. However, there are three compelling reasons to treat code as a (logical) black box: First, performance assessment is often delegated to non-developers: colleagues from computing centers, centralised HPC specialists, or external service providers. If we want to discuss performance analysis methodology, it is counterproductive to assume in-depth code knowledge. Many people conducting assessments cannot be experts in every code they examine. Second, requiring code expertise within assessment teams would imply that core developers are always available. This might be an ideal in the spirit of eXtreme Programming [1], where the customer must be accessible at all times. It is unrealistic in science, where PIs are busy with various tasks, developers frequently leave projects—as they graduate, accept new positions, or are reassigned—and where code components are often written by individuals and are so complex that only a few people understand the core parts. Finally (and most importantly), knowledge about a codebase tends to bias the assessment.

Developers inevitably make assumptions about reasons for code behaviour when working on performance analysis. (Senior) Developers tend to "defend" their code, explaining any flaws as natural consequences of their domain's challenging nature, their awareness of future requirements, or their in-depth knowledge of other setups not covered by the present configuration. Simultaneously, we have frequently observed junior developers downplaying or challenging existing code—likely to prove themselves by pointing out "how things really should work"—and consequently interpreting every piece of data through this lens. This list is not comprehensive. The whole team dynamics dimension deserves further discussion (in Section **??**). For the time being, we conclude that having emotionally invested developers involved in early assessment is counterproductive due to their inherent bias.

Work in progress/todo The link to team dynamics is broken as this part is work in progress and not yet written.

Assessment crime 2 *People conduct assessments as code experts. Due to this, they deliver biased assessments since they think they already know the code's performance characteristics and its flaws without an objective assessment of its "real" behaviour.*

For a high-quality assessment, it helps to step back and (pretend to) know nothing. This way, we avoid entering performance analysis with bias. Adhere strictly to the observational perspective. Even if it is your own code, try to detach yourself and pretend you have no prior knowledge. Abstract.

How it works

- ⊞ Pretend you know nothing about the code. Blend out your expert knowledge and start on a fresh sheet.
- ⊞ Ignore what others, notably developers pretend to know or have found. This makes your assessment biased.
- ⊞ Force yourself not to do any tuning before the assessment and analysis are complete (within reason).

How it does not work

- ⊟ Start with the attitude "I know what this code does and where there are weaknesses".
- ⊟ Involve someone who's emotionally involved in the code's development into the reporting. They will always defend their brain child.

2.3 Fundamental HPC and assessment terms

We try to keep the HPC jargon in this write-up to a minimum. You might argue that this is a bad idea as we want to run scientifically sound assessments. However, we have to acknowledge that most scientific code today is written by people without a computer science background, and that most scientists do not have an extensive HPC training. It is therefore not only a matter of accessibility of the performance analysis and assessment itself to try to avoid too much jargon—if we use too much of it, we run the risk that our outputs, i.e. the assessments themselves, cannot be read and interpreted appropriately by our target audience. After all, most of the assessments are not done for ourselves, but for other scientists.

At the same time, performance analysis tools are written by computer scientists, software stacks are written by computer scientists, and computer hardware is built by computer scientists. It is hence clear that there's a lot of jargon buried in any conversation around performance. We have to find a healthy middle ground to make precise, in-depth assessments and analyses, without immersing ourselves in technical fuzz.

Some of the most important performance analysis terms When we speak of *profiling*, we mean recording and presenting (accumulated) metrics. The classic example

is measuring how much time we spend in a function or waiting for a message to arrive. *Tracing* is the counterpart to profiling. Here, we record performance data of individual events. We are interested in the sequence of operations occurring within the system. Profiling focuses on the effect of program execution while tracing focuses on the source of behaviour (Figure 2.1). Many developers confuse these definitions, which we took from the VI-HPS community.

Tracing can certainly be used to generate profiles, provided the recorded performance data encompasses all quantities required for a particular metric. Conversely, the reverse problem is ill-defined: it is generally impossible to reconstruct a trace from a profile without specialized knowledge of the code's internal workings. One might immediately ask why we don't trace all the time. The answer is equally simple: It is too expensive. Tracing generates enormous amounts of data. Therefore, all real-world tracing relies heavily on filtering, where you determine beforehand which types of events to trace, which quantities to measure, and which program parts to monitor. Consequently, we revise our introductory statement: most traces are *not* well-suited for deriving program metrics, as they are, by definition, incomplete.

To create a profile, we distinguish between two techniques: We can *sample* program execution. Essentially, we run the code, pause it at regular intervals, and each time examine our metric of interest at that point. Over time, we gather a good impression of which functions are called most frequently, how many MFlops/s we achieve, and so forth. Some people refer to this process as monitoring. As with all statistical methods, the data quality improves the longer an application runs. However, the risk of missing important but brief effects remains.

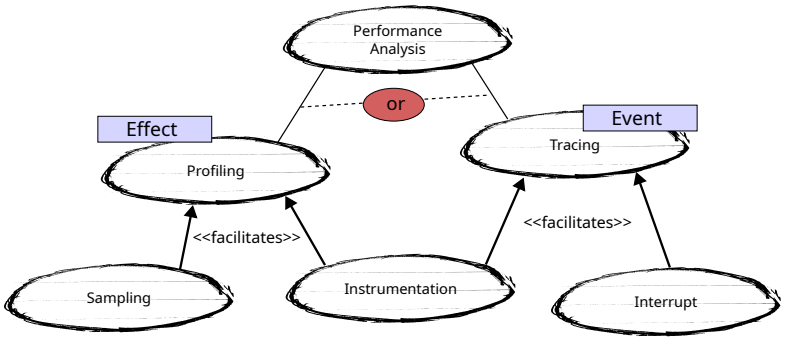


Figure 2.1: Overview of some fundamental performance analysis concepts.

Alternatively, we can *instrument* an application. Instrumentation involves inserting measurement code into our actual code which then measures quantities of interest whenever these insertion points are reached. We can insert either at compile time, or in a postprocessing step after the compile, or alter the code as we go. Instrumentation yields very accurate data (provided we instrument all relevant code sections), but it can become prohibitively expensive.

Most functions in our applications call other functions. Therefore, we must always clearly state whether any metric we present (e.g., runtime) is *inclusive*, comprising all

contributions from called routines, or *exclusive*, representing only the function itself.

When tracking metrics, measuring time and call frequency is straightforward: we either measure system time or simply count occurrences. Other metrics are more complicated to obtain. Fortunately, modern systems offer performance counters. They are special registers within the chip that automatically count certain events. For example, they can be configured to count every floating-point operation or every main memory access, allowing us to sample them or read their values upon completion. Performance counter registers are expensive to manufacture, so they are limited in number, and not every vendor tracks all possible events. Consequently, some analyses might not be possible on every system, and complex analyses might require multiple code executions, as only a few event types can be measured per run.

As we know about the existence of hardware counters, it is clear how sampling and tracing can obtain machine data beyond mere timing and call counts: They simply monitor (read out) the counters. Through this, they can make statements about the number of floating point operations completed, the number of cache misses, and so forth. For many of these concepts, we can invert the control flow: Rather than polling a counter or quantity, we can define *interrupts*, i.e. situations in which the program flow is stopped and we call our profiler or tracer. Interrupts can be realised through hardware or software. Their beauty is that the performance analysis sits somewhere outside of the core program logic—most of the time, our code runs totally unaware of the analysis in the background.

Hardware jargon We revise the bare minimum of hardware jargon here to get started, and then refer to further resources to revise details.

Supercomputers consist of many different individual computers linked through a high-performance interconnect (network). These computers are called *nodes*.

Each node is a full-grown computer and hence hosts many individual *cores*. All the cores on a node share their memory (but cores from different nodes do typically not share memory). Technically, many vendors call their cores *hardware threads*, as a core is a technical building block and might host multiple threads. We use hardware thread and core as synonyms.

On larger systems, you do not use the compute nodes directly. Instead, you log into a *login node*, you compile your code there, and then you hand your code over to a *scheduler*. The most popular scheduler today is Slurm. Slurm takes your request, i.e. how many nodes you want to have and for how long, as well as all the other requests from all other users and puzzles out a fair and efficient way to assign the resources. For performance analysis, it makes sense to investigate if your system of choice offers a test or benchmarking queue with a high priority. These queues usually offer only a few nodes and restrict the maximum application runtime. In return, jobs handed over to these queues are scheduled with high priority.

Compilation should be on the login node, as the actual compute nodes might not provide all the required software, and many production nodes are also shielded from the Internet, i.e. even updating you repository is a pain. Benchmarking should not be done on the login nodes, as you share them with all other users. In contrast, you can tell the scheduler that you want to have nodes *exclusive*, so no other user interferes with your

measurements. This is particularly important for all data stemming from performance counters, as these counters are a hardware feature that we read out, i.e. our profiler or tracer cannot distinguish if a hardware counter's data stems from your code or another code if multiple users coexist on one machine.

Core performance quantities The runtime of a code is written here as $t(p)$ where the p is either the number of nodes or number of cores. It depends on the context. The *speedup* of a program is defined as

$$S(p) = \frac{t(1)}{t(p)}, \tag{2.1}$$

which is typically smaller than or equal to one and tells us how much faster we made an application by using p compute units (cores or nodes) rather than only one.

The *parallel efficiency* is given by

$$E(p) = \frac{S(p)}{p}. \tag{2.2}$$

It quantifies to which degree adding further compute resources was worth it.

2.4 Bottom-up vs. top-down assessment

Reader's guide

It might be better to skip this section and to read it after the first high-level assessment is completed.

Work in progress/todo This part is yet to be written and will basically point out how the POP methodology differs from what we do here.

2.5 Symptomatic vs. causal assessment

Reader's guide

It might be better to skip this section and to read it after the first high-level assessment is completed.

Work in progress/todo This part is yet to be written and will basically point out how the POP methodology differs from what we do here.

Performance assessment can be guided by the observed behaviour or by monitoring the machine. One might refer to this as a symptomatic vs. causal assessment. Scientists will always favour a causal assessment where a hard metric can be used to explain certain observed behaviour. However, this is not always possible and frequently not the most economic way to assess a code's performance. Notably in early phases of a performance analysis, it makes sense to focus on what is observed without digging into the reasons for some behaviour.

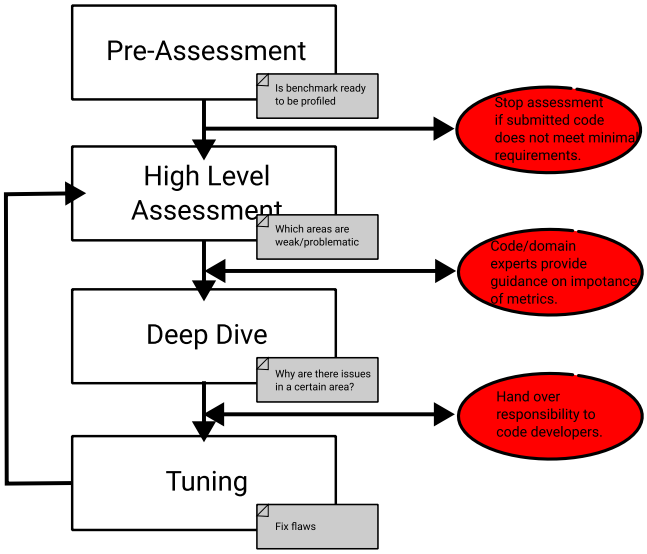
Part II

High Level Overview

Chapter 3

Process

The guidebook proposes a straightforward four-step assessment workflow:



1. The *pre-assessment* (Chapter 4) determines whether the code is ready to be assessed. It verifies that the code installs without expert knowledge, exposes the required configuration knobs, passes basic sanity checks, and meets minimum requirements. It validates that it offers a well-suited benchmark. This phase is carried out by the benchmarking team, but it is the code authors' responsibility to ensure all requirements are met. If they are not, the assessment terminates: an assessment team cannot work productively continue.

2. The *high-level assessment* (Chapter 5) examines five areas of interest independently, identifying where the code performs well and where problems arise. Studying the five areas separately has drawbacks, but sharpens the focus of

subsequent steps. This phase is conducted by the assessment team without involvement from the code developers. However, its findings must be interpreted and weighted collaboratively with the developers, who also indicate whether further in-depth analysis is warranted.

3. Once areas of concern are identified, the *in-depth assessment* (chapters from Part III) drills into specific weaknesses to diagnose why the code underperforms. This phase is again conducted independently by the assessment team, informed by the feedback gathered in the previous step.

4. The findings feed into *code tuning*. This falls within the broader scope of performance engineering rather than the core assessment, and should ultimately trigger a re-assessment of the codebase.

3.1 Goals

The phased process is built around three core principles. They shape the workflow above.

- **Objectivity**. The assessment must be objective and free of bias. This requires a strict separation between the roles of assessor and code owner (developer), even in cases where the same person occupies both roles.

- **Autonomy**. Assessments can be conducted in close collaboration with developers or kept entirely separate. The POP methodology, for example, requires strong involvement from code owners. By contrast, EuroHPC technical assessments for resource allocation aim to minimise such involvement—the assessor should require no knowledge of the codebase. The present methodology follows the latter tradition.

- **Responsibility**. The process assigns clear ownership over who delivers what, and when. Defined check-in points require developers to deliver specific inputs, while deliberately excluding them from other phases—keeping the workflow productive for the assessment team and assigning the ownership of the analysis to the assessment experts.

Further to that, we introduce a two additional conventions:

- **Specificity**. The assessments are machine-specific, benchmark-specific and configuration-specific. They make no attempt to generalise for the code over multiple machines, for different benchmark settings, or even only configurations of certain (hyper-)parameters such as GPU warp sizes, cores per rank, or MPI buffer settings. This means that all statements refer to the specific test setup. They can make statements along the lines "this code scales poorly for this particular configuration of cores per rank", but it is not possible to derive further, deeper insight; although it is obviously possible to compare the same assessments for the same code for multiple machines with each other, or to derive improved system setups from the analysis.

- **Hypothesis-driven**. The assessment follows certain universal guidelines, but the subsequent deep dives, i.e. the actual performance analysis is strictly hypothesis-driven. Once we start to explain why a code behaves in a certain way (cf. Section 2.1), we start with a hypothesis, evaluate this hypothesis and then either confirm it and dive deeper or discard it and study the next hypothesis.

3.2 Criticism

Assessment crime 3 *The roles of performance analyst and performance engineer are not carefully separated.*

There are at least two alternative schools of thought when it comes to performance analysis. The the first school emphasises that constructive performance improvements can only result from a dialogue between performance expert and code owner. They point out that performance assessment or analysis is difficult if not impossible if there's no context on the code's structure, algorithms, data sets, objectives, and so forth.

Our strategy appreciates the need for a feedback cycle, but is anxious of the bias introduced by a very close collaboration. We have seen multiple cases, where code owners almost expected the performance analysis to "tell me what I want to hear": they did steer the analysis down a certain route, and expected instructions how to improve the code parts they had been interested anyway rather than an objective assessment of the overall "product".

This is the reason why our document advocates for a strict separation of the role of the assessor and the code expert. That does not mean that the assessor may not know anything of the code and should not speak to the code owner. It also does not mean that these two roles always have to be separate people. One person can switch hats. It however does ask for a strict separation and the attempt to avoid that any code knowledge influences the assessment. Again, the analogy to a GP helps: A good GP will always ask the patient where it hurts, but they will be very critical when they are told about self-diagonses (guided by Google).

A second school of thought is strongly metrics- or data-driven. Here, the idea is that it is key to gather as many metrics as possible initially and to distill hypotheses and eventually insights from these data. The approach is popular with tool developers.

Our document advocates for a hypothesis-driven workflow, where a hypotheses or metric of interest determine what data is of interest. Once this is clear, we suggest that assessors pick a tool to gather these data—often a different tool per purpose. Both approaches, hypothesis- and data-driven come along with pros and cons. In practice, many performance analysis will employ a hybrid. However, we favour the workflow and sequence of hypotheses guiding what data is gathered in this write-up.

3.3 Shortcomings

Like any process, the current approach carries trade-offs.

First, the strict separation of roles and the emphasis on autonomy demand a certain level of code maturity: the code must be usable by a non-expert with little to no prior

knowledge of the codebase or domain. In particular in agile projects in scientific computing, where the user base is often very small and highly specialised, these risks are particularly pronounced. While the process excludes developers from the majority of the assessment, it requires strong commitment from the code owners to deliver a ready-to-use benchmark in the first place.

Second, the reliance on reports as the primary output also creates a risk of conflict, as developers may dispute findings and bristle at how they are presented. This is particularly important in scientific code development projects, where code authors develop a strong emotional attachment to code, since they build their career on it. A successful delivery of (multiple) code assessments requires a constructive collaboration. The fact that the process tries to be as objective and therefore somtimes might yield brutal insights or performance statements, challenges constructive collaboration. This has to be taken into account (and moderated) by the involved senior managers.

Third, the attempt to isolate performance flaws and metrics bears risk to separate flaws that are strongly correlated. For example, poor core utilisation on some cores might lead to a situation where the overall load balancing appears to be weak. The assessment handles these metrics independently and hence runs risk to report balancing issues rather than the cause of the problem. This is an example where domain and code expertise can help, but it remains a fundamental drawback baked into the process itself.

Finally, both the role separation and the exclusion of optimisation work create a risk of losing expert knowledge embedded in the code or domain. This tension is addressed in the wrap-up discussion (Section 5.7), which acknowledges that independent assessments can struggle to connect findings to scientific requirements. Prioritising steps, metrics, and studies nevertheless remains difficult. The present approach always risks trading independence and objectivity for deeper insight.

Chapter 4

Preparation

Reader's guide

This section is absolutely key before you start an assessment!

In this chapter, we discuss various steps that we want to do *before* we dive into assessing a code's performance. It summarises activities that ensure that we are well-prepared to run assessments that yield meaningful outcomes. We typical refer to this stage as a 'pre-assessment', i.e., we work through a series of steps to document our work from choosing/receiving a characteristic benchmark of a codebase, through to verifying that we can run the benchmark. We see this as a necessary barrier of entry because a code submitted to an assessment service should be documented sufficiently such that the analyst can initially treat the code as a black-box, and get setup almost 'without thinking'.

There is a checklist in Appendix B which summarises some of the key information you might want to ask from a code before you start any benchmarking or analysis. In line with the present section, this checklist is high level, but may provide an idea of the minimal information an assessor requires about a code before we get started. It is almost a customer-facing summary of this chapter's content.

4.1 Benchmark characteristics

Before we start a performance assessment, we need to agree on what type of measurements to perform: There has to be one well-defined benchmark. If we are given a set of different setups and try to compare them, this will make our task complicated. We run the risk of comparing different characteristics produced by different code parts.

- This one benchmark has to run without any user interaction. In the normal case, the sponsor has to provide a script which builds and runs the whole benchmark "as a black-box".

- It has to be very easy to scale that benchmark up and down, i.e. to make the problem solved bigger or smaller[1].

- The benchmark has to be a simulation run which is representative of a real-world run. It might be significantly smaller and shorter, but it should reflect the core characteristics. If the benchmark does so is domain knowledge, i.e. an assessor has to rely on community and expert feedback to ensure this property holds.

- The benchmark may not run for too long, and it may not terminate too early. As a rule of thumb, the "best case" run (say a whole node) should take approximately a maximum of 10 minutes. By best case we may mean a parallel run, which could mean that a serial run of this benchmark takes significantly longer.

None of the properties are surprising: We need a well-defined toy setup, and we have to be able to modify its size to make statements on its scalability. In this context, it is important that we know exactly how changes of input sizes affect the compute workload (Section 4.4).

If domain experts tell us afterwards, i.e. after we have completed our assessment—they will always do that afterwards when they are unhappy with the outcome—that this particular run is not characteristic for larger runs, as it lacks a feature or does not really use a particular routine heavily, our whole assessment exercise is corrupted. And with it the whole assessment will be a big disappointment. It is their job to pick a setup that represents what we are interested in and is nevertheless reasonably small.

Having a "small" setup that completes in reasonable time is important since some performance analysis tools induce significant overheads, i.e. make the code significantly slower. If our vanilla run already needs more than an hour, we will never get an answer on time. Furthermore, a lot of tools gather significant amounts of data per second. Long-running simulations thus run the risk that our analysis tools fail to handle the amount of data collected. Training material from VI-HPS, e.g., recommends around 10 minutes, though the precise value is up to (the patience of) the assessor. When colleagues submit their code, they typically provide a reference configuration using more than one core. We will later reduce the available resources artificially to obtain insight into the code's behaviour. If the baseline already runs for 10 minutes and we take a quarter of the resources, such experiments quickly become infeasible.

Two more properties are important:

- It has to be clear how to disable I/O for the benchmark. This topic is covered by Section 4.5.

- The baseline configuration has to consume a reasonable amount of memory, but may not fill out the whole memory either.

[1] For classic PDE solvers or molecular dynamcis simulations, this is easy: you increase or decrease the mesh points or particles, respectively. For AI workloads, we can manipulate batch sizes for example. There are simulations, where the problem size is invariantly tied to the problem to be solved, i.e. we cannot simply make it slightly bigger or smaller. In this case, this item has to be taken with a pinch of salt and corresponding metrics and measurements have to be adopted or skipped.

If a setup is too small—notably if it fits into the local caches—we will not obtain realistic data, as critical system components that quickly develop into bottlenecks are not used. We provide remarks on machine properties in Section 4.2.1. If a setup is too big however, we again run risk that there is not enough slack for tools to run additional statistics on a program run. A memory footprint of 20%–80% of the main memory is reasonable. Attempting to keep the runtime down, we typically operate closer to 20%, as long as this is significantly more than the L3 cache and significantly more than the GPU's largest cache. While the ways to extract system properties are discussed in the aforementioned Section 4.2.1, there are different tools to record a code's memory consumption:

Linux command line

Linux's `/usr/bin/time` offers the option to track the maximum memory usage[a]. Adding the `-v` flag yields additional information which delivers, depending on the system configuration, information on the maximum memory envelope. The quantity `Maximum resident set size` is the quantity of interest.

[a] Typically does not work if you use the terminal builtin `time`, i.e. a call without the absolute path.

Linux command line

Linux comes along with different on-board tools to track the memory consumption of a code, so assessors can validate any statement on the actual memory footprint. A simple `top` displays the memory usage and can be used to extract all relevant information (there are more sophisticated variants such as `atop` or `htop`, but `top` is available on each and every system).

- Log into the compute node of interest, i.e. the node on which a code runs. With Slurm, `squeue` provides you with the node name, while your admin team has to allow `ssh` into the node.
- Combine the call with `-b -d <duration>` to run `top` in sampling mode.
- Pipe the outcome into a text file.

Valgrind

The Valgrind tools perhaps provide the gold solution to monitor memory usage. They can be used a simple prefix to the program call:

```
valgrind --tool=massif ./code
```

yields all the data require to get an overview of a code's memory needs. Different

to `top` and `time` it however introduces (potentially massive) runtime overhead. At this point, a simpler monitoring call might be more appropriate.

Checklist

1. Your benchmark compiles as a black box, i.e. you can give it to another person together with a `README` file and there might be a few instructions that they have to copy-n-paste into their terminal, but other than that there's no other preparatory work to be completed.
2. Your benchmark runs without any user interaction.
3. Your benchmark finishes within less than ten minutes for a "best case" configuration.[a]

[a] To analyse different aspects of the code, we do not need to rely on the same benchmark. For example, when conducting scaling analyses, there may be performance characteristics that are only revealed at high core counts. Hence, for some codes we may set up our benchmark to have runtimes exceeding 10 minutes at low core counts, such that the problem size is large enough to see high core count scaling effects. However, at high core counts we would expect to see this benchmark runtime sit within this 10 minute upper bound.

No matter how we create this setup, we have to be able to run all experiments with absolutely minimal user interaction. If we have to run through a complex pipeline for each individual experiment, we will not be able to measure anything productively.

How it works

- ⊞ Ask the code owner ("assessment customer") to provide you with a ready-to-use benchmark.
- ⊞ Validate that this benchmark meets the criteria outlined in this section. Otherwise, return it immediately.

How it does not work

- ⊟ Try to make a code run and design a benchmark yourself. This is the job of a domain/code expert.
- ⊟ Expect insight from benchmarks that somebody without domain expertise has chosen.

4.2 Working environment

The setup of a working environment can have a big impact on a software's performance. Clearly the hardware has a large impact, but so can the software environment not to mention the structure of the cluster, e.g., the wait times on partitions can impact the turnaround time for the assessment.

In this section we lay out some features of the working environment that we believe are important when setting up the working environment for code compilation and

benchmarking, starting with the machine. The hardware details are crucially important as we may later on wish to do comparative benchmarking on different hardware, or the code submitter may have requested a hardware platform that they have access to or are interested in.

4.2.1 Machine details

Modern supercomputers are very complex and it is virtually impossible to memorise all the hardware details of a testbed given the zoo of processor types, generations and flavours. While it is possible to consult your compute centre's webpages or to consult your system administrator, it might be more reasonable to obtain that data directly from the machine.

Assessment crime 4 *Never grab the specification from the login node. Instead, ask for an interactive terminal on a compute node and get the data there. Many supercomputers' login nodes are slightly different to the compute nodes, i.e. slightly different make, more memory, more cores, . . . So you might end up with the wrong hardware specification if you analyse a login node.*

Linux command line

On every Linux system, information about the CPU on the system is held in a central file. You can read it out by typing `cat /proc/cpuinfo` into the terminal.

Likwid

If you have Likwid installed, you can invoke `likwid-topology` to obtain in-depth information about your CPU.

MAQAO

In MAQAO, you find some topology information in the menu rubric *Topology*.

In the checklist below we recommend some hardware features to note:

Checklist

1. Number of nodes
2. The node layout, e.g., dual socket of AMD EPYC 7702 64-Core Processors
3. Caches (including sizes and associated cores) and memory
4. NUMA domains

How it works

- ⊞ Gather the exact spec of your testbed under realistic load. If this information is already available (e.g. due to previous assessment exercises), it makes sense to reuse it.
- ⊞ Note down any features of the testbeds queues and partitions, including which are most likely to be used for the assessment.

How it does not work

- ⊟ Throw advanced tools onto your code and expect that they report the theoretical machine capabilities themselves.

4.2.2 Software environment

The software environment, e.g., compilers, libraries, modules, etc. can have a significant impact on a code's performance. We do not yet worry about the impact that these dependencies have on the code's performance, as by this stage we are simply interested in building the code. However, as with machine details, it is very useful to have the software environment documented.

Modules

To grab the loaded modules and dependencies (following the code's documentation) we can simply

```
module list
```

How it works

- ⊞ Following the documentation, load in relevant dependencies and note them down using the `module list` command including any version numbers.

How it does not work

- ⊟ Copy across the dependencies as they appear in the documentation.

4.3 Compiler setup

When worrying about optimisation, many modern compilers can leverage significant optimisations for us. Likely, performance analysts will receive codes that specificy build instructions that include a variety of compiler flags with some level of optimisations. It may be the case that the build instructions are old and so we could exploit further optimisations.

At this stage, we keep things simple. We simply observe what we have received, either through a submission or through code documentation, and note it down. This is a

crucial stage, because at this point we really see whether we are able to follow the code owners' instructions and reproducibly build the benchmark. One thing which should stand out at this stage is what parallelism is included in the build process. Are we using OpenMP flags? Are there flags for targeting device architectures for GPUs? We take note of these as these will be relevant later in the assessment. We will go into more details about compiler optimisations in Section 7.1.

One thing we we might want to include at this stage are debug symbols. These instruct the compiler to insert additional information into the source code which tools can then later use to draw inference from, e.g. which source code line a certain instruction stems from. Usually, you cannot know from a single instruction in a machine code, where this comes from in the original code. With the debug systems, we inject exactly this information into the executable. Therefore, we should add `-g`. Some compilers support more detailed information (e.g. using `-g3`). It requires studying the compiler's feedback whether these flags disable optimisations, in which case we might have to balance insight vs. best-case speed. Often profilers require debug symbols to allow them to relate machine code to source which help when digging into the performance of the generated machine code.

Checklist

1 Your benchmark is compiled with debug symbols (`-g`).

How it works

- ⊞ Follow the proposed build instructions and document them, noting anything that faults.
- ⊞ Ensure to document flags that relate to the parallelism, such as OpenMP flags.

How it does not work

- ⊟ Take the build instructions and then aim to implement further compiler optimisations.

4.4 Code scaling and complexity

Reader's guide

The complexity discussion is mainly relevant when we conduct an MPI (weak scaling) assessment. In many cases, we might be able to skip it initially.

We have to know or have to be told the computational complexity of the underlying algorithms. If we have a pure Monte Carlo setup, then doing 100 times more trials should last around 100 times longer. If we have a parabolic Partial Differential Equation (PDE) solver with explicit time stepping and a regular grid, we have to know that the

problem sizes increases by a factor of 2^d if we half the mesh spacing. However, these algorithms also have to reduce the time step size, so if we measure over a fixed time span, we actually will have four times more time steps and therefore increase the workload by a factor of 2^{d+2} once we half the mesh size. On the other hand, if we run an Adaptive Mesh Refinement (AMR) code for a hyperbolic setup with explicit time stepping or a Smoothed-Particle Hydrodynamics (SPH) code, we need some kind of output that tells us exactly how many particles or mesh cells we have and what time step sizes we have picked. In this case, time step sizes go down linearly with mesh spacing. At the end of the day, we are interested in the *throughput* of the system, i.e. the time we need to update one quantity of interest. It is this metric that we will start from in the next steps.

Finally, the code owner should supply the analyst with some understanding of how to scale the problem. In an overly simplified manner: 'what can I vary to double the problem size?'. However, this raises the final point which is: we need to document the mechanics of configuring the scaling. Aside from the algorithmic complexity we need to know: are we tweaking command line arguments? Or editing a configuration file? This can seem simple, but for the analyst who is not familiar with the code, this is not always obvious.

4.5 Memory, storage and I/O

String operations are excessively expensive. Hence, where possible, string operations should be minimised by the code's developers, and our assessments should use a version of the code which does as few string operations as necessary. In this context, it is important that there's no console I/O. We don't want to study complex outputs; we are busy enough with handling the output of performance analysis tools.

Assessment crime 5 *The code writes a lot of information to the terminal.*

More importantly, if our code writes excessive amounts of data, we almost certainly profile the I/O efficiency rather than the runtime behaviour. Unless we want to explicitly study I/O, we are well-advised to switch off any file outputs, database outputs, and also excessive file reads which will not compromise the benchmark being representative of a normal run. A log file alone can become problematic when these files become suddenly huge or are updated frequently. In the best case, a performance assessment benchmark writes and reads barely any data at all, i.e. it is largely hard-coded and silent.

Assessment crime 6 *The code reads or write to files excessively.*

Some codes rely on significant input data, or can write large files. This can be particularly true of machine learning benchmarks which may require large training datasets. It is good to note down these approximate file sizes as this can impact our working environment, e.g., having to work in a different file system on the cluster. In keeping with looking at the disk footprint of the benchmark, this can also be a useful stage to note down the memory usage, as in line with Section 4.1, as by this point we should have a running test case of the benchmark.

Checklist

1. Your benchmark does **not** write (a lot of) files.
2. Your benchmark does **not** write a lot of information to the terminal.

How it works

- ⊞ Disable I/O, exclude reading and writing files from any assessment as far as possible (and as long as you don't want to benchmark exactly this I/O).
- ⊞ Remove terminal dumps. If the developers want to preserve the output (e.g. for validation or debugging), suggest to embed them into macros which can be compiled out of the code prior to production runs.
- ⊞ Note down rough file sizes for input and output files

How it does not work

- ⊟ Start with a high-level assessment straightaway. In particular any shared memory measurement will be corrupted by excessive terminal dumps.

4.6 Wrap-up

By this point we, as the analyst, should be able to comment the following

- What does our working environment look like?
- Can we build the code and run a test case?
- Does the test case take a "reasonable" amount of time? Following our guidance in Section 4.1.
- Does the test case use a "reasonable" amount of memory? Again, following from Section 4.1.

At this point we really have the evidence as to whether we can proceed with assessing the code. Again, we treat this pre-assessment step as a barrier of entry. If we are not able to reasonably build and run the code — even potentially with communication with the code owner — then we cannot make any meaningful progress with the assessment.

Running through these steps we have likely begun to notice some features of the code, in particlar, what parallel model does it use? By building and/or running the test case we see whether there the code uses MPI, OpenMP, GPU offloading, or a combination. These features of the parallelism build into the next step in the assessment: the high-level assessment.

Chapter 5

High-level first glance

There are various ways or strategies to obtain a first high-level overview and how to get started if we want to understand a code's performance better. Every team, every project, every scientist have their own approach, and it often changes from project to project. In this chapter, we propose a workflow running along few very simple principles:

1. We study the performance characteristics for *different machine part/aspects separately*.

2. We use a simple *blackbox performance metric* per dimension to find out first if there are issues or not.

3. We are not interested in explanations of *why* we see any performance flaws. We simply document *what flaws* we observe.

Per dimension, we later break all performance flaws found down into subrubrics, and dig deeper and deeper into the code. Along this way, we construct working hypotheses of why the code performs poorly. But this comes later. For the time being, we focus in a bird's eye view, which is an analysis which we can conduct, for example, with a simple script, a few tools and mere timings.

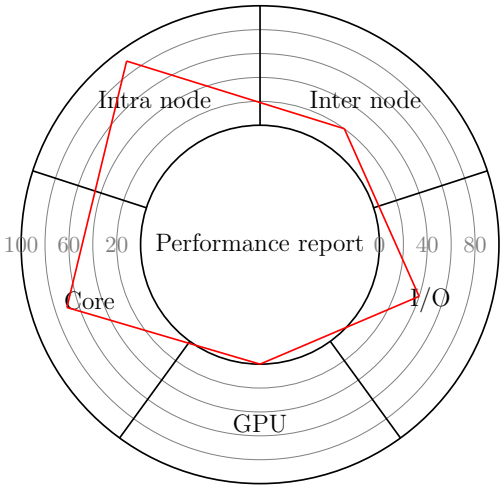
Assessment crime 7 *Modern preformance analysis tools (e.g. Intel Performance Application Snapshot (APS) and Linaro MAP) and methodologies offer a high-level overview as kind of an automatic service. This is a very valuable feature, as it provides you with a quick overview of your code's characteristics. While we appreciate these opportunities, we are however not sure if this is really the right starting point for performance assessment. In our approach, we do not use one single run and its characterisation to get started, but we use many different runs (with different configurations) to zoom into particular feature dimensions such as intra-node scaling and see if there are issues there. Instead of one global overview, we obtain a set of metrics which we combine into an application profile. We do not rely on a* **one run uncovers it all** *approach.*

5.1 The assessment rubrics

We classify codes along five different rubrics:

1. core,
2. GPU,
3. input/output,
4. intra node, and
5. inter node.

For each of these rubrics, we initially aim for a single three-valued: All is fine (green), there may be issues (yellow) or there are obvious flaws (red). In line with our overall presentation approach, we use thresholds of 60% and 80% to distinguish these classes. It is convenient to present the overall data via a spiderweb diagram (below for a code that has I/O issues, doesn't scale particularly well between nodes, has no GPU yet reasonably good core performance and node scaling):



The idea to look at the five different performance dimensions separately is natural yet does not come along without shortcomings. Many performance issues within a code affect multiple dimensions. Think about a poor GPU utilisation that materialises in bad load balancing as well. When we dig down recursively into finer and finer metrics later on, we have to take such cross-influence into account. Eventually, we might have to return to our highest level of assessment again and again.

We take a characteristic benchmark which is representative of a typical code workflow and are inkeeping with recommendations from Section 4.1. Typically this means we have a benchmark that demonstrates the parallel behaviour of the code and characteristic I/O. Since we treat the code as a block at this stage, we give no thought to

the setup of the benchmark. We receive a benchmark configuration from a developer directly, or we follow recommendations from the documentation.

As we analyse the dimensions of performance characteristics independently, the order of the analyses therefore does not really make a difference. As flagged throughout Section 3, this approach helps to manage complexity but has intrinsic weaknesses in itself.

How it works

- ⊞ Ignore most things you know about tools to avoid that you are not able to see obvious shortcomings.
- ⊞ When you report on your findings, focus exclusively on what you observe. Avoid any attempt to interpret things.

How it does not work

- ⊟ Throw a huge, powerful tool onto your code and expect it to flag problems immediately.
- ⊟ Start to measure without a clear plan of what you want to measure.

5.2 Core performance

Summary

We consider a code to make efficient use of a compute core's capabilities if it uses more than 80% of the compute facilities that an artificial benchmark would be able to exploit. We consider a code to make reasonable use of a compute core's capabilities if it uses more than 60%.

For the core-level analysis, we examine how efficiently your code harnesses the chip's microarchitecture. Does your code leverage the most advanced instruction sets available? Are we maybe even compute-bound? Does it properly utilise the memory interconnect? Are we memory-bound? Today's chips come equipped with powerful vector computing capabilities and sophisticated cache and streaming features designed to move data smoothly between memory and compute registers. Our analysis evaluates how effectively your implementation takes advantage of these capabilities.

To capture core behaviour at a high-level we simply want to measure the code's peak performance relative to the maximum of the hardware. Tools can tempt us to reach immediately for a roofline model, or potentially even more detailed metrics around cache misses, vectorisation ratios, etc. These topics are relevant to a core performance analysis, but we want to start high-level or we otherwise risk focusing on an individual metric which obfuscates or distracts our analysis. By focusing simply on peak performance, our initial core performance investigation should sit us somewhere in the 'traffic light' table below.

5.2.1 Benchmark preparation

Before beginning our data collection we want to ensure that our data is a reliable and accurate representation of the peak, single-core performance of the code.

Assessment crime 8 *We assess a setup that fits completely into one of the caches.*

Measure the total memory footprint of your code and ensure that the working set exceeds the largest cache you find on your system (cmp. remarks on caches below); techniques for making these measurements are given in Section 4.1.

Not all benchmarks have a single-core implementation, and indeed in some cases a single-core run is not useful. For example, some benchmarks are implemented to be 'full node'. Hence, we again can use an indicative run of the code, e.g., if the code submitter has suggested a 'full node' run then we can perform a high-level analysis averaged across all cores. We will cover the details of this practice in the next section.

To some degree, the argument holds the other way round for the cores: Modern chips are often not capable of cooling all of their cores if all of them run at full compute capacity. They therefore downclock the system when they encounter lots of compute-intense operations. As we only assess scalar peak performance in this first exercise, this effect should not arise or be negligible. You might however want to double-check.

5.2.2 Data Collection

Once we have prepared our benchmark we are now ready to begin making measurements, and the data we need for this metric are:

- The maximum single-core, peak performance of the hardware
- The obtained single-core, peak performance of our executable

As mentioned above we can do this either by using a single-core run or by using a parallel run and taking the average of these values across the cores. As we are only making one observation, it is advised to use exclusive compute resources (i.e., an exclusive node here) to get the most reliable data that is not impacted by other job allocations.

We also advise taking these measurements with high thread affinity, in which the software threads are pinned to physical cores. If the threads have low affinity this can damage performance hence we would ideally like to see the core performance in its 'best light', i.e., ruling out a simple performance hit. An example of how to do this for the high-level core analysis is given below, but more detail on affinity and allocation of threads, etc. is laid out in Section 6.3.

To make these measurements, we must turn to a performance tool. Multiple performance tools are able to make these measurements, and we list below basic instructions of how to perform these measurements with some of our 'go to' tools.

Likwid

To measure the peak performance of a hardware example with Likwid it is only a few lines

```
likwid-bench -t peakflops -w S0:16kB:1
```

This command pins the `peakflops` microbenchmark to a single core on the first socket (labelled `S0` here), to ensure maximum thread affinity. We use a convention advised by the Likwid developers in which the `peakflops` microbenchmark is performed with an input dataset half the size of the L1 cache. For example, in the commands above we use a 16kB input which is half the size of a 32kB L1 cache (which we can find by running the `likwid-topology` command). As we scale the `peakflops` microbenchmark across a node, we scale up this data input size by the number of cores, e.g., if want to run this analysis across 128 cores we use an input of 128×16kB, giving

```
likwid-bench -t peakflops -w N:128*16kB:128
```

though we stick to just single-core performance for now.

The instruction set usage is something we can evaluate per core. Depending on the character of the code, we might want to pick a more advanced instruction set (e.g. `peakflops_sse` or `peakflops_avx_fma`). As long as we argue about the overall code performance, we find the plain peak performance to be absolutely sufficient. Being within 80% of that one for the total program execution is typically a good sign.

Once we have the thresholds, the code's performance itself can again be used by multiple tools. Again, likwid is our personal favourite:

```
likwid-perfctr -f -C 0 -g FLOPS_DP ./code
```

which provides you with the information relevant, although the exact names of the groups might differ from system to system. In this example we have pinned the serial run of the code to the first physical core, and have measured the double-precision floating-point operations.

5.2.3 Evaluation

Once we know the maximum performance and the obtained performance, we can compute their ratio, i.e., the normalised peak C_{core}. This metric is a little bit weird (and we will revise it later), as we typically normalise against the scalar peak performance. This is surprising given that all mainstream CPUs support vectorisation and hence can deliver several times more than that. However, we empirically found that starting with a comparison against the baseline is a good starting point: Few codes spend all of their runtime exclusively in very compute-heavy routines. Therefore, there will always be code parts which do not exploit vectorisation. If those that are compute-heavy do so and hence lift the average total peak into the regime of the scalar peak, this is already an overall efficient code.

Performance metric (rule of thumb) 1 *The delivered MFlops/s normalised against the scalar peak is a good high-level indicator for the compute efficiency.*

Peak FLOPS proportion	Description
$C_{\text{core}} \geq 0.8$	Core compute performance is good.
$0.8 > C_{\text{core}} \geq 0.6$	Core compute performance is not effectively using the hardware to its maximum potential.
$0.6 > C_{\text{core}}$	Core compute performance is poor.

If $C_{\text{core}} \geq 0.8$, the code uses the compute capabilities (very) efficiently. If $0.6 \leq C_{\text{core}} < 0.8$, the code does not use the compute units particularly efficiently and we have to investigate further.

How it works

- ⊞ Run a standard benchmark to determine a realistic maximum peak performance per core.
- ⊞ Assess your code to find out if it makes sufficient use of the available compute units.

How it does not work

- ⊟ Apply some performance counters as a black box without a focus on peak performance.
- ⊟ Jump straight for a tool which gives significant data on vectorisation, cache misses, etc.
- ⊟ Immediately dig into any performance flaw without evaluating the other four rubrics first.

5.3 GPU performance

Summary

We consider a code to make efficient use of the GPUs if it achieves more than 80% of the theoretical peak performance of the GPU. We consider a code to make reasonable use of the GPUs if it achieves more than 60%.

Similar to the core performance, the GPU analysis examines how efficiently your code harnesses the GPU. Our analysis evaluates how effectively your implementation takes advantage of the extreme parallelism of the GPU.

To capture GPU behaviour at this high level, we want to measure the peak performance relative to the hardware's capabilities. As with core performance, the tools can draw us towards more detailed metrics and models, but to minimise distractions and clarify our analysis we restrict this rubric to the proportion of peak performance.

5.3.1 Benchmark preparation

We want to ensure that the data we collect about the code is a reliable and accurate representation of the performance of the code on a single GPU. To this end, we require that the code is built for the specific GPU architecture, and that the benchmark is larger than the GPU cache - see assessment crime 8. You should measure the memory footprint of your benchmark on the GPU in order to verify that the benchmark is large enough.

Assessment crime 9 *We study a code that is not built against the latest version of CUDA/HIP/etc. supported by the hardware*

For example, if you are working with an Nvidia H200 GPU, which supports CUDA compute capability 9.0, you must use a cuda toolkit version of at least 11.8 and specify `-code=sm_90`.

5.3.2 Data Collection

Each manufacturer of GPUs provides their own management software suite, which is capable of collecting similar but slightly different sets of metrics. Below are some instructions for Nvidia hardware to extract the metrics necessary for the high-level assessment (instructions for AMD and Intel are to be included in the future). However, if you are using other types of accelerators or accelerators from other manufacturers the software suite required to collect these data may be significantly different or incapable of providing the recommended metric.

In order to calculate the FLOPS/s, we generally need to collect metrics of the total number of floating point operations executed.

Nvidia NSight

On Nvidia GPUs using Nvidia NSight Compute (`ncu`), the metrics required are in the format `sm__sass_thread_inst_executed_op_{METRIC}_pred_on.sum`, where `METRIC` is `dfma`, `dadd`, `dmul`, `ffma`, `fadd`, `fmul`, `hfma`, `hadd`, and `hmul`.

We can request this set of metrics using a regex, and so profile the program:

```
ncu --metrics "regex:^sm_.*d_op_(d|f[^p]|h).*\\.sum$" --csv \
    --log-file out.csv ./code
```

We also need to measure the execution time of the program without the profiler attached - here we make the assumption that the number of operations executed is the same in every run regardless of execution time.

Where multiple different precisions are used, they will run at different rates on the hardware, but we need a single number to compare against the theoretical performance of the GPU. We therefore count each double-precision instruction (those beginning with `d`) for double the number of operations, and each half-precision instruction (those beginning with `h`) for half the number of operations. Additionally, because each `fma`

(fused multiply-add) does two operations in one instruction, we further double the number of operations these instructions count for.

We finally calculate the observed work rate:

$$R_{\texttt{observed,FLOPS/s}} = \frac{\sum N_{\texttt{opers}}}{T_{\texttt{exec}}} \tag{5.1}$$

5.3.3 Evaluation

Performance metric (rule of thumb) 2 *The percentage of the theoretical maximum GPU FLOPS/s throughput achieved by the computational kernels. This may also be referred to as GPU peak-performance-percentage.*

We calculate the peak performance ratio as

$$C_{\text{GPU}} = \frac{R_{\texttt{observed}}}{R_{\texttt{theoretical}}} \tag{5.2}$$

GPU peak performance ratio	Description
$C_{\text{GPU}} \geq 0.8$	GPU compute performance is good
$0.6 \leq C_{\text{GPU}} < 0.8$	GPU compute performance is not particularly good. This could become a significant issue with other GPUs including newer generations.
$C_{\text{GPU}} < 0.6$	GPU compute performance is poor.

How it works

- ⊞ Compile your code with the latest version of CUDA/HIP etc. supported by your GPU's architecture
- ⊞ Normalise the floating point operations for programs where multiple precisions are used

How it does not work

- ⊟ Run a benchmark that is smaller than the GPU's cache
- ⊟ Dive into other metrics recommended by the analysis tools
- ⊟ Focus on GPU performance without evaluating the other relevant metrics in the high-level rubric

5.4 I/O performance

Summary

We consider a code not to suffer from IO constraints if it spends less than 20% of its runtime in I/O.

5.4.1 Benchmark preparation

When benchmarking I/O, or benchmarking with I/O enabled, we advise using a benchmark with a typical I/O load - this should be verified by the code's developers. It can tempting to reduce the I/O operations to 'game' this metric, but if a code does rely on heavy and frequent I/O then this should be included in the characteristic benchmark.

5.4.2 Data Collection

There are relatively few tools that focus on I/O performance. Hence, we are restricted in the number of tools that can collect our metric of interest, namely, the proportion of runtime spent doing I/O operations. We list below an example of how to do this with a Darshan, an I/O focused performance tool.

Darshan

In this example we present paths and modules as they are defined on a local system. Any user of this book would therefore have to find the relevant Darshan files on their system of choice.

For a dynamic runtime analysis we setup Darshan with

```
export DARSHAN_DIR=/apps/developers/tools/darshan/3.3.1/1/

intel-2021.4-intelmpi-2021.6 # path to Darshan
export DARSHAN_LOGDIR=./darshan_logs
mpiexec -genv LD_PRELOAD ${DARSHAN_DIR}/lib/libdarshan.so
-np 4 ./code
```

in which the `darshan_logs` directory is user defined for hosting Darshan profile outputs. The logging environment variable name can be different so make sure to run

```
darshan-config --log-path
```

to find the name on your system. Once this has created a `*.darshan` binary file this can analysed via

```
darshan-parser --total my_log.darshan
```

The relevant metrics from this output are in the form

```
STDIO_F_*_TIME: cumulative time spent in different
types of functions.
```

Therefore depending on different forms of I/O — e.g., `POSIX`, `MPIIO`, `STDIO`, `H5F`, `H5D`, `PNETCDF_FILE`, `LUSTRE`, `DFS`, `DAOS` — we need to sum together the read and write times.

5.4.3 Evaluation

Performance metric (rule of thumb) 3 *We evaluate*

$$C_{I/O} = 1 - \frac{t_{read} + t_{write}}{t_{total}}$$

where t_{read} and t_{write} are the times the code spends in reading and writing data, respectively, whilst t_{total} is simply the total runtime (inclusive of t_{read} and t_{write}).

I/O Proportion	Description
$C_{\mathrm{I/O}} \geq 0.8$	I/O does not dominate runtime.
$0.6 \leq C_{\mathrm{I/O}} < 0.8$	I/O forms a significant part of the total runtime.
$C_{\mathrm{I/O}} < 0.6$	I/O dominates the runtime.

How it works

- ⊞ Run an indicative I/O workload and observe time spent in reads and writes
- ⊞ Accumulate time from different forms of reads and writes into a single I/O runtime
- ⊞ Observe how much of total runtime is spent in this I/O runtime

How it does not work

- ⊟ Run your benchmark without any indicative I/O workload
- ⊟ Dig into the details of the code's I/O from tools like Darshan, and begin to draw conclusions

5.5 Intra-node (node-level) performance

Summary

We consider a code to make good use of the cores on a system if it can use more than 80% of its cores with more than 80% of (strong scaling) efficiency. We consider a code to make reasonable use of the cores if it can use more than 60% of the total core counts with more than 80% efficiency.

On an individual node, our first goal has to be that we use all the hardware threads efficiently. Some people prefer the term cores or speak only of threads (even though we might want to distinguish the logical from the physical thread concept). The term intra-node hence means all those cores sharing one memory. Hereby it does not matter if we have a code using MPI or a genuine multithreading language: We assess the runtime characteristics for a code physically sharing its memory. It might be written logically in a distributed memory fashion.

Our method of choice to argue about node-level performance is strong scaling. In principle, we ask ourselves "what would happen if we had only one core on our node" (we use the term core and hardware thread as synonyms from hereon). If our code

needs 80 cores on one node, then we would expect that it only needs 40 cores on two, and 20 cores on four. This is obviously an artificial question to ask: Why would we not use all the node? Despite the fact that we have the cores available anyway and therefore might want to use them, finding an answer to the "what if" questions helps us to uncover inefficiencies. How much performance do we leave on the way as we use more and more cores? How much more computation should we be able to squeeze out of the machine?

The underlying performance model is called Amdahl's law, and we will return to this law in a future chapter. For the time being, we can abstract from any performance model or law, and simply focus on the scaling behaviour over cores, as we have defined it in Section 2.3: We alter the number of cores available to our code until we have reached all the cores available, i.e. use the whole machine, and track the time to solution dependence on core counts.

5.5.1 Benchmark preparation

Throughout this assessment, we use one fixed problem setup. The setup has to be chosen such that we can compute it on a single core if necessary. However, it also has to be reasonably big such that it makes sense to exploit all threads of a node.

Assessment crime 10 *We start an intra-node assessment with a code which only uses a tiny fraction of the main memory.*

We may assume that a node and its memory are somehow balanced. If we don't use a reasonable amount of this memory for a test run, then there is simply not enough work to do to utilise all cores. We can barely blame the code. Along with previous recommmendations, we should have a memory footprint which is at least 20% of the total RAM available on the whole node (cf. Section 4.1).

5.5.2 Data collection

For the data collection, we first run the simulation without any parallelisation, which is, trivially, a single-core run. This gives us t_{serial}. After that, we switch to the parallel code version and vary the core count p_{intra}. This gives us $t_{\text{intra}}(p_{\text{intra}})$.

Assessment crime 11 *Use a code base that is relatively inefficient on a single core and use this as baseline for any scaling claims.*

It is important that we question the code developers on different code variants prior to the assessment: If they offer, for example, quite sophisticated task-based parallelism for shared memory, we still should calibrate all runtime against a serial code without these tasks. That is a fair benchmark that avoids any tasking overhead where it is not needed.

Consequently, $t_{\text{serial}} \neq t_{\text{intra}}(1)$ since we use different executables—once compiled with support for shared memory and once without. The two measurements will differ and we can assume $t_{\text{serial}} \leq t_{\text{intra}}(1)$. If the code is not available without any parallelisation, use $t_{\text{serial}} = t_{\text{intra}}(1)$. After that, we collect the remaining $t_{\text{intra}}(p_{\text{intra}})$ with the parallel code variant.

Assessment crime 12 *We do not use the compute resources exclusively. For example in SLURM, clusters often have exclusive queues, or the* `--exclusive` *flag can be passed.*

OpenMP

In OpenMP, we can simply set `OMP_NUM_THREADS` to achieve this. After that, we run our code and should automatically get the correct core count. Depending on your system's configuration, you might want to set some additional command line variables such as

```
export OMP_PROC_BIND=close
```

or similar to ensure that the operating system does not swap your threads around, i.e. decide at one point to migrate a thread to a different core. This never gives good performance. Be careful as well with setting the OpenMP threads manually while using another core specification technique such as Slurm's configs or tasksets (both below): If you do so, you might force all the OpenMP threads onto a few cores only, i.e. accidentally over- or undersubscribing the testbed.

Slurm

Using Slurm, we do not have to set environment variables ourselves, instead Slurmmanages resources and so we can request a quantity of cores with the `-c` flag, e.g.,

```
#!/bin/bash

#SBATCH -c 2

./code
```

or equivalently use `#SBATCH --cpus-per-task 2`.

Linux command line

The one thing that works always is using

```
taskset -c 0-3 ./code
```

to employ, e.g., the first four cores only. However, you should be careful when you use this in combination with Slurm or OpenMP which also make choices on cores. You might end up with competing or contradictory settings and end up

with a suboptimal system usage. We also faced some issues with TBB using this approach.

MPI

If we have a code base which employs MPI only, we set the number of ranks per node through the `mpirun` call or the scheduler (Slurm) environment.

For hybrid MPI+X codes, we have a certain freedom: We can use an arbitrary number of MPI ranks—indeed our analysis also works for pure MPI codes as long as we stick to one node—and use threads on top such that we exploit all p_{threads} hardware threads of interest. Picking one rank and four threads is, logically, equivalent to picking two ranks and two threads each. It is part of the assessment to play around with these degrees of freedom. Once again: If we spot sudden runtime jumps (yellow dots in example sketch) and if these jumps coincide with a NUMA domain or socket, then this might be a strong indicator that we should use MPI instead of threading. However, one might argue that this is already part of an in-depth analysis rather than a high-level overview.

We recommend that for a code of this form: when working with a researcher or developer, it best to request a typical configuration that they may use. They may be able to advise on their experiences, and from there we can then perturb about this configuration.

Likwid

Likwid has the in-built `likwid-pin` tool for handling thread-affinity for a specified number of cores. Cores can be requested explicitly, i.e., specific NUMA domains, sockets, etc. can be requested. This gives the analyst great flexibility in the choice of cores across the node. The Likwid call typically resembles

```
likwid-pin -c 0-...
```

but you can also provide a comma-separated list of specific cores. Using Likwid's pinning with OpenMP means that there's no need to set `OMP_NUM_THREADS` anymore—the pinning yields properly set OpenMP settings implicitly.

If you have the opportunity to mask out the initialisation time and to focus only on the "real" runtime of your benchmark code, use it at this point. Notably when you work with very short execution times, the initialisation overhead might otherwise dominate your measurements.

5.5.3 Evaluation

Our roadmap to determine how good the shared memory behaviour is, i.e. our strategy to come up with a single number, is straightforward. First, we calculate the code's

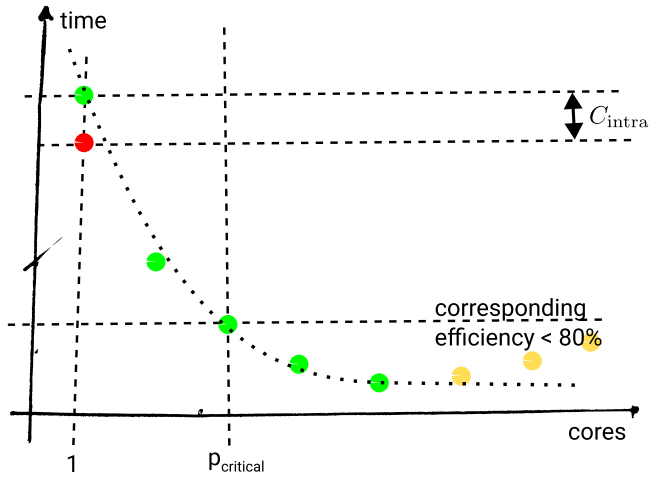


Figure 5.1: A scatter plot tracking the (strong) scaling data feeding into our intra-node analysis. The red node is the code's runtime without any parallelisation enabled, while the green and yellow measurements stem from the parallelised code over various cores. For our high-level analysis, we are interested at which point (core count) the resulting parallel efficiency falls below an acceptable threshold.

parallel efficiency. We use a slightly modified version as compared to Equations (2.1) and (2.2), where we calibrate the runtimes against the serial code runtime:

$$E_{\text{intra}}(p_{\text{intra}}) = \frac{t_{\text{serial}}}{t(p_{\text{intra}}) \cdot p_{\text{intra}}}$$

is normalised against the serial runtime without any parallelism support. We track how much we sacrifice by introducing the parallelism in the first place, which is typically a better indicator for inter-node performance flaws than the textbook efficiency definition.

Second, we determine the number of cores for which our parallel efficiency falls below the thresholds 80% or 60% respectively (Figure 5.1):

$$\begin{aligned} p^{80\%}_{\text{critical,intra}} &= \arg\max_{p_{\text{intra}}} \{E_{\text{intra}}(p_{\text{intra}}) \geq 0.8\} \\ p^{60\%}_{\text{critical,intra}} &= \arg\max_{p_{\text{intra}}} \{E_{\text{intra}}(p_{\text{intra}}) \geq 0.6\}. \end{aligned}$$

It is not unheard of that codes yield data where the runtime starts to raise again once we exceed a certain core count (yellow dots in Figure 5.1). This means that the speedup deteriorates completely. The above metric tries to flag early if we run into such a regime or also into one that leads to a stagnation of the runtime and hence to an efficiency decline.

Finally, we introduce our performance metric. It normalises the critical thresholds against the maximum number of cores or threads $p_{\text{max,intra}}$ that we have on our system.

Performance metric (rule of thumb) 4 *We evaluate*

$$C^{80\%}_{intra} = \frac{p^{80\%}_{critical,intra}}{p_{max,intra}} \qquad \textit{and} \qquad C^{60\%}_{intra} = \frac{p^{60\%}_{critical,intra}}{p_{max,intra}}$$

to determine how well the code scales over a given node.

We use the two metrics to determine if we consider the code to have an intra-node flaw.

Efficiency	Description
$C^{80\%}_{\text{intra}} \geq 0.8$	Shared memory scaling is good.
$C^{80\%}_{\text{intra}} < 0.8 \wedge C^{60\%}_{\text{intra}} \geq 0.6$	Shared memory scaling is not particularly good, and we notably might run into problems with the next generation of chips where the core count will increase.
otherwise	Shared memory scaling is poor.

How it works

- ⊞ Compile your code without multithreading/parallel support and compare its performance to a code without any parallelisation to determine the overhead of the parallel implementation.
- ⊞ Run a strong scaling study and compare the efficiency to prescribed thresholds how many cores you want to the system to use reasonably efficiently.

How it does not work

- ⊟ Skip intra-node scalability studies, as the code is "only" MPI. If a code is only MPI, you should still do strong scaling tests on a single node, where MPI physically uses shared memory.
- ⊟ Perform a sole strong scalability study without taking the implementation penalty into account.
- ⊟ Jump straight into a further "in-depth" analysis without evaluating the other high-level metrics.

5.6 Inter-node performance

Summary

We consider a code to make efficient use of multiple nodes if it scales weakly. In particular, we consider it to be scaling if it recovers from inefficiencies (efficiency drops below 80% or 60%) significantly whenever we increase the problem size, i.e. "gets back" to (almost) 100% or 80% efficiency, respectively.

Inter-node behaviour means that the code runs over multiple physically separated nodes. In an HPC context, this almost always implies that it uses MPI. While MPI

employs a programming model which treats memory physically and logically as separate and makes them exchange data via messages, there are (software) solutions that pretend to the code to have one large shared memory over many nodes. Such virtually shared memory solutions would fall into this assessment category as well. Conversely, one can use message passing, i.e. logically separate data spaces on a single node offering one physically memory. This would fall into the intra-node assessment rubric from Section 5.5.

Our method of choice to argue about inter-node performance is weak scaling, but we do not really (yet) dive into a proper version of Gustafson's law [2]. Instead, we simply start from one reasonably big setup which would fit onto a single node and scale it up over multiple nodes. This setup's parallel efficiency will go down as we increase the number of nodes, as communication overheads kick in and the work per node reduces. Once the efficiency falls under 60%, utilisation is really poor. Therefore, we increase the problem size and see if we can get the efficiency up again. The increase should lift the efficiency significantly over 80% again. If it does, we end up with a zig zag pattern, where we test until we hit the 60% threshold, increase the problem size (and hence lift the efficiency above 80%) and then increase the node count again. The code struggles to scale weakly, if the subsequent problem size increase improves the efficiency, but cannot push the efficiency above the 80% efficiency threshold anymore. The code does not scale at all, if the problem size increases fail to keep the efficiency above 60%.

5.6.1 Benchmark preparation

For weak scaling, it is absolutely key that we first identify a performance metric that is complexity-invariant: Let an algorithm's compute cost scale in $O(n^g)$ with the problem size n. If $g = 1$, the algorithm has linear complexity, for $g = 2$ we have quadratic complexity. If we double n, the time we need for a computation on a given number of nodes p_{inter} should double, but if and only if the algorithm is linear. If is quadratic, it will become four times bigger. As we want to compare the times (and later efficiencies) over different problem sizes, we have to calibrate accordingly.

The challenge is that the speedup formula

$$E_{\text{inter}}(p_{\text{inter}}) = \frac{S_{\text{inter}}(p_{\text{inter}})}{p_{\text{inter}}} = \frac{t(1, n_1)}{t(p_{\text{inter}}, n_{\text{inter}}) \cdot p_{\text{inter}}}$$

contains timings determined with (potentially) two different problem sizes. If the algorithm's complexity is known, and as long as we assume that the runtime constants, i.e. all the overheads incurred, are kind of invariant regarding the problem size, the calibration is easy:

$$E_{\text{inter}}(p_{\text{inter}}) = \frac{t(1)/n_1^g}{t(p_{\text{inter}})/n_{\text{inter}}^g \cdot p_{\text{inter}}}. \tag{5.3}$$

where the employed timings are raw timings.

Assessment crime 13 *Benchmark assessments try to argue about a code's efficiency without knowing the code's computational complexity.*

When preparing a benchmark for the inter-node, weak scaling analysis, a key point for the performance analyst is that we must have some understanding of how to scale the problem size, i.e. how to change the simulation problem size, but also the code's complexity g.

5.6.2 Data collection

Amending the complexity

For some codes, the code owners can easily clarify what g is. For others, the arising g might not be a sole polynomial, in which case we have to amend (5.3). It might also depend on factors that are difficult to fix a posteriori and need some output analysis:

Time stepping codes sometimes amend the time step size dynamically. If the problem size changes, also the chosen time step sizes change, and we have to calibrate the timing accordingly: In this case, we are interested in some timing per time step, i.e. the fact that a bigger setup picks more time steps should be factored out.

Other codes employ dynamic mesh refinement. That is, the number of compute elements changes over time. If we increase the admissible number of elements by a factor of four, this might not necessarily lead to an exact increase of the workload by a factor of four. Instead, we have to parse the outcome and normalise the timings by actual cells.

A final example are solvers for non-linear equation systems: As we change the problem size, a solver might suddenly need more or fewer iterations. We are interested in computational throughput here rather than mathematical efficiency, and hence once again have to parse the number of actual iterations and calibrate all measurements accordingly. Due to dynamic termination criteria in the underlying linear algebra, the resulting calibration might be highly non-linear.

Memory footprint

Another consideration for weak scaling is the actual problem size:

Assessment crime 14 *We scale up the problem such that the individual work per nodes becomes very small.*

As a rule of thumb, we can simply track the memory footprint per rank and ensure that it stays within a certain regime such as 60% of the available memory per node.

Hybrid codes

If a code supports MPI+X with X being some kind of multithreading or has no multithreading at all, then the baseline has to be chosen such that we employ all cores of one node. That is, the baseline $t_{\text{inter}}(1)$ might already use more than one MPI process, as long as all of these processes are pinned to the same node.

Workflow

Once we have the algorithmic complexity and memory footprints under control, we can conduct the actual measurements:

1. Pick a problem size n_1 for one node.
2. Measure the time $t_{\text{inter}}(1)$.
3. Increase the node count n_{inter} and measure the timings. This is basically strong scaling in line with Section 5.5.
4. Compute the computational efficiency on-the-fly in line with (5.3).
5. If the efficiency $E_{\text{inter}}(p_{\text{inter}}) \leq 60\%$, increase the problem size n_{inter} and continue the measurements from hereon with the increased problem size.

The recipe leaves it open by how much we should increase n_{inter} every time. A rule of thumb is that we maximise the value such that the nodes still can handle the problem, and, at the same time, try to ensure that each compute node hosts local subproblems with roughly the same memory footprint that we obtained for n_1 on one node.

5.6.3 Evaluation

Our roadmap to determine how good the distributed-memory behaviour is, i.e. our strategy to come up with a single number, is straightforward. First, we record all the p for which we had to increase the problem size into a set $\mathbb{C}^{60\%}_{\text{critical,inter}}$. For these problem sizes, we only pick the timings/efficiencies for the bigger setup, i.e. after we have increased the problem size.

Performance metric (rule of thumb) 5 *We evaluate*

$$C^{80\%}_{inter} = |\{p \in \mathbb{C}^{60\%}_{critical,inter} : \quad E_{inter}(p) \geq 80\%\}| \qquad \textit{and}$$
$$C^{60\%}_{inter} = |\{p \in \mathbb{C}^{60\%}_{critical,inter} : \quad 80\% \geq E_{inter}(p) \geq 60\%\}|,$$

to evaluate to which degree the efficiency curve remains above 60% *or within the corridor of* 60% − 80%.

The assessment now evaluates if the majority of the jumps manage to push the efficiency above 80% or at least manage to keep it consistently high (Figure 5.2). Let $C_{\text{inter}} = |\mathbb{C}^{60\%}_{\text{critical,inter}}|$ count the number of jumps. If all jumps remain above 60% with the majority above 80%, the scalability is very good. If most jumps remain above 60%, the scalability is acceptable.

Efficiency	Description
$C^{80\%}_{\text{inter}} \geq 0.8 \cdot C_{\text{inter}} \ \wedge \ C^{80\%}_{\text{inter}} + C^{60\%}_{\text{inter}} = C_{\text{inter}}$	Distributed memory scaling is good.
$C^{60\%}_{\text{inter}} \geq 0.6 \cdot C_{\text{inter}}$	Distributed memory scaling is not particularly good.
otherwise	Distributed memory scaling is poor.

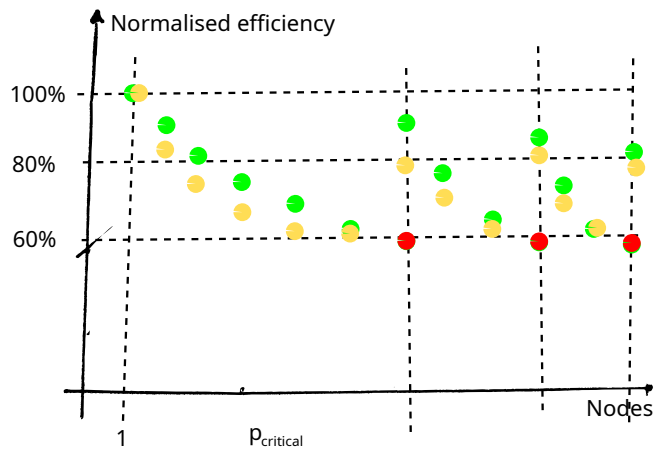


Figure 5.2: Efficiency data for two codes (yellow and green). At three node counts, their efficiency underruns 60%, so we increase the problem size before we continue. The green code always recovers from the poor efficiency once we increase the problem size, but we struggle to push the yellow curve above the 80% mark again.

How it works

- ⊞ Run the code over an increasing number of nodes and increase the problem size every time the efficiency becomes inacceptably low.
- ⊞ Study if these problem size increases help you to push the efficiency up significantly.

How it does not work

- ⊟ Run the same problem size over a few different node counts and make statements on the code's behaviour.
- ⊟ Rely on raw timings that do not take the computational workload per setting into account.

5.7 Wrap-up

Summary

Interpreting high-level assessment outcomes requires input from code and domain experts.

Any high-level analysis should be wrapped up and summarised by the assessor. Producing visual artefacts helps to convey key insights effectively. The outcome metrics are objective in the sense that they represent observed code behaviour. There is no room

for evaluation or contextualisation. Nevertheless, this summary must be written and shared with care. Also dealing with feedback and comments requires guidelines.

Challenging the outcome Claims of code exceptionalism must be challenged directly: If developers argue that their code is so complex and specialised that poor performance on certain metrics is natural and expected "for this particular field", that claim can be noted but must not influence the assessment or its publication. The assessment reflects the state of the code. If developers instead argue that the code behaves differently in practice, this reveals that they have failed to provide a characteristic, meaningful benchmark—not that the assessment is wrong.

In both cases, such pushback is a warning sign and grounds to consider terminating the assessment process. As both cases are not unheard of, it makes sense to agree on a code of conduct prior to the start of the assessment. This includes an agreement of the code owner to study the outcome and provide (constructive) feedback.

Interpreting the data The five metrics provide raw data, and it is not clear how 85% in one rubric relates to 85% in another rubric. Correlating and weighting the five high-level metrics therefore requires domain/code experts' input, and it is impossible for the assessor to have knowledge about this. Once the data are available, it is the domain and code experts who have to assess, with the additional knowledge the assessor cannot have, how the metrics should be balanced to wrap up the high-level assessment. Notably, whenever the high-level assessment is used to prioritise follow-up studies, the assessor must consult code and domain experts to determine where to focus.

For example, a code exhibiting both poor core performance and poor I/O behaviour does not necessarily warrant tackling both issues: the code community is best placed to judge which flaw has the greater impact on their science. The metrics presented in this chapter carry no relative weighting against one another.

Domain experts may also identify correlations between metrics. Poor intra-node load balancing, for instance, will often lead to poor average core utilisation as a direct consequence. An expert can recognise such dependencies and rightly argue that follow-up work should target the balancing issue first. This matters especially because the high-level assessment treats the code as a single block, without distinguishing between individual execution phases. Domain experts are therefore well placed to clarify why certain flaws become particularly significant at scale—for example, in larger runs or longer calculations.

Part III

Deep Dive

Chapter 6

Workflow

Before we start to dive into a certain are of interest, it makes sense to narrow down the search space, but it also makes sense to briefly discuss how to navigate through the plethora of options that one might want to study.

6.1 Localisation

So far, we have assessed our code as more or less one big black box.

- We did not ask *which part of the code* causes a problem.
- We did not ask *when* it arises throughout the execution.
- We did not ask *where*, i.e. on which rank (if we run over multiple nodes) a problem arises.
- We distinguished, to some degree, *what* we want to study in our code, i.e. which quantity of interest.

The importance of the four dimensions strongly depends on what we are interested in and what we are investigating: Single-core flaws imply that the "where" question does not play a significant role. Inter-node flaws might suggest that the "where" question is the primary concern to address. Not all four dimensions are always equally important.

6.1.1 Tool support

To assess the code and dive deeper into details, it makes sense to zoom into details along these four dimensions to manage complexity. For this, most tools provide features. We broadly distinguish two approaches: Tools allow us to *explore and zoom* into data after the collection, i.e. to explore the gathered data space, while *filter mechanisms* allow the assessor to record only specific information right from the start. Obviously, an a posteriori approach is superior in the sense that we do not run risk that we "forgot" to record data initially. However, the sheer amount of data recorded by many tools often make it necessary to define filters beforehand.

Filtering

To filter data, most toolchains provide marker APIs. Alternatively, they come along with precompilers which take your code and add the filter on/off statements to it before it is passed on to the compiler. Some very popular tools are sketched in Chapter 9, including the APIs they offer.

Zoom in

VTune

VTune offers in almost all tabs an overview over the temporal behaviour at the bottom which splits up the observations over cores. A right click or drawing an observation interval allows you to zoom in.

6.1.2 Labelling of code parts

Software typically contains quite a lot of boilerplate code: predominantly administrative routines that enable calculations but don't deliver actual science flops. For several follow-on exercises, it will be very useful to know which code parts deliver science and which code parts do not directly add scientific value. This is domain knowledge. Using it contradicts, to some degree, our ambition to run a performance assessment in a black-box manner.

However, we do not ask an assessor to understand what a code does. We require the assessor to know which code parts contribute towards the science case. For this reason, it is absolutely sufficient to hand a first profile back to a domain expert and to ask them to label those routines that do the actual calculations.

As an example: In a linear algebra code, the actual matrix-vector products, scalar products, ... all deliver science. The allocations for temporary block matrices, the exchange routines for rows of the matrices, any indexing over the sparsity pattern, and so forth do not contribute calculations. They are absolutely essential, but they do not give us the MFlops/s.

As your analysis progresses, you might run into situations where functions suddenly appear to be hot that you have not seen before. In this case, you can kind of safely assume that these functions do *not* contribute to the science directly. If they didn't show up for a small-scale, few-core run, then they likely do not feed directly into the result.

How it works

- ⊞ Upon your very first profiling, ask a code expert which code parts (functions) actually perform calculations, i.e. contribute towards the science.

How it does not work

- ⊟ Assume that all routines deliver science (even though you may assume that all are necessary).

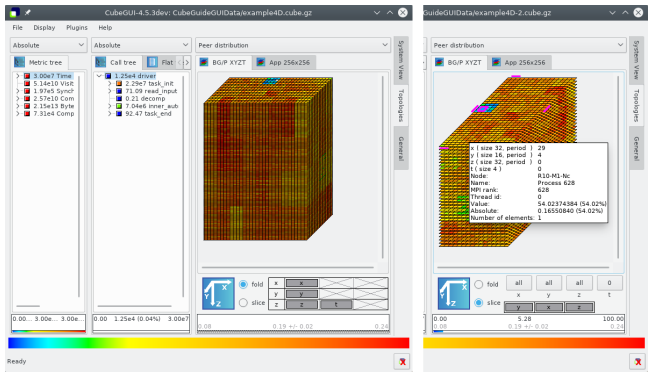


Figure 6.1: Screenshot of the CubeGUI from the tool's webpage.

6.1.3 Related work

Our four rubrics are strongly inspired by the CubeGUI work[1]: In this graphical front-end, which is used, among others, by the Scalasca tool, users can study effects along the dimensions metric, call tree/graph, and system (Figure 6.1).

The metric corresponds to the hypotheses discussed in the following deep-dive sections. They are picked up in the subsequent section, too. If you plan to study a particular hypothesis which corresponds directly to a CUBE metric, the tool becomes particularly useful. call tree/graph is a the so-called program dimension and allows users to study where certain flaws result from, i.e. which code parts cause the pain. The system dimension finally helps users to filter along the where dimension.

We propose to extend the three-dimensional analysis from CubeGUI by a temporal dimension for most analyses, which is implicitly supported by Cube by filtering data production a priori or by embedding certain code parts into markers such that the active code sections can be distinguished.

6.1.4 Take away

The most frequent quantity of interest (metric) to developers is typically how much time is spent in routines and which routines or function calls are to be blamed for some behaviour. The routines consuming most time are called *hotspots* of a code. This section's rationale is that hotspots are important, but simply looking at a profile and picking out the most expensive routine does not provide all data required for HPC workloads. Massivel load imbalanced for example will manifest in totally misleading results on undersubscribed nodes, while the profile on the oversubscribed node will maybe not even suggest that it causes issues.

This chapter thus makes a few simple suggestions:

[1] https://helmholtz.software/software/cubegui

1. Focus on the smallest relevant time scale of a program run (e.g. study only one iteration of an iterative solver),
2. narrow down the amount of code you consider (i.e. look at the top hotspots only),
3. measure only on a few characteristics nodes or cores,
4. and always consider the effect first.

That is, we suggest to invert the analysis flow: We define which effect we are interested in, and narrow down the search space taking the effect of interest into account.

How it works

- ⊞ Once you have a first high-level overview, always use a tool to find out if the effects of interest are spatially and temporally localised.
- ⊞ Focus as much as possible.
- ⊞ Adopt the search space to the effect you are after, i.e. the hypotheses you are studying.

How it does not work

- ⊟ Start with some hotspot analysis.
- ⊟ Jump straight into some traces or profiles without spending time to think what metrics and program parts are relevant to answer your hypotheses regarding performance flaws.
- ⊟ Draw conclusions from global observations on all code parts and compute resources (cores or ranks).

6.2 Navigation

There is a plethora of specific phenomena for each of the five rubrics of interest that one might want to study. Often, they align with a performance hypothesis such as "the throughput per core might be low, as data never resides in the L3 cache". In this case, one would study the L3 cache access characteristics as phenomenon. Running through different phenomena or hypotheses in an anarchic way following intuition is often not leading to the correct conclusion, as it runs risk that important details are missed out. While the following sections—one for each of the five fundamental assessment dimensions—each host sets of effects to dive into, an overarching challenge is how these effects, aspects and things to study are properly structured. For example, should we first study the bandwidth and then the latency behaviour before we look at vectorisation, or the other way round?

We originally intended to create something like a decision tree: a unique, clear, deterministic guideline of which phenomena to study one after the other with the goal to come up with a clear explanation of why a code behaves the way it does. Such a tree yielding a cascade of hypotheses to follow would provide a guideline that reads similar to: First look at the load balancing. If the load balancing is bad, fix it; otherwise have

a look at the number of synchronisation points. Then continue with false sharing. And so forth.

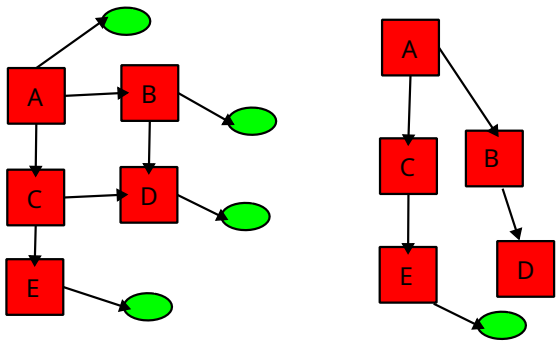


Figure 6.2: Left: In the example, we have a graph over hypotheses or effects (red squared) `A`–`E` that we might want to test for and that depend upon each other. Some hypothesis evaluations suggest certain explanations for the code's behaviour (green ellipses). In most cases, diving into one phenomenon will however require studying another follow-up hypothesis. As we work our way through the graph of hypotheses excluding more and more potential explanations, we build up a tree of hypotheses (right). This tree is analysis-specific, i.e. might look different for the next case study.

Initial assessments with trial trees made it clear to us that such a decision tree is very difficult to construct. Indeed, it is not clear whether a universally acceptable hypothesis tree to assess performance does exist. Different people have different ideas and preferences of what such a tree should look like, and these should be accommodated to keep motivation high and to allow people in the way that is most effective and constructive to them. Further to that, it might be reasonable and valuable to use a tailored tree per assessment exercise and code: every code is different, so the best-case sequence of effects to analysis is different, too.

Therefore, we propose to arrange performance hypotheses to study in a directed graph (Figure 6.2). It is the assessor's duty to navigate this graph to find explanations. For each code assessment, the analysis journey might yield a different traversal of the graph. Implicitly, each assessment will yield a decision tree informed by the graph: We run through a sequence of hypotheses. For each hypothesis, there might be multiple options where to go next, and the assessor picks one. That might lead into a dead end, so we return to an earlier hypothesis that we didn't follow-up immediately.

It does not matter in which order we run through the graph of tests and hypotheses. However, it matters that we stick to their systematic order and do not jump into an arbitrary node right from the start. If we do this, we might miss out of context and alternatives; we work unscientifically.

How it works

⊞ Navigate through the graph of phenomena and hypotheses one by one.

- ⊞ Keep track of the effects analysed and record the resulting decision tree.

How it does not work

- ⊟ Aim for a decision tree that can universally and mechanically be applied to all benchmarks.
- ⊟ Pick different phenomena to look at randomly without following the causal dependencies encoded within the decision graph.

6.3 Affinity

6.4 Process placement

Chapter 7

Core

7.1 Compiler optimisations

Before we continue, it is important to double-check whether we have used the correct compiler settings. It makes no sense to benchmark a code which is not producing tailored code for our particularly machinery (therefore it is important to benchmark on the right system), and it makes no sense to benchmark a code that is not using the latest compiler features.

First of all, we work with the the most aggressive optimisation that our compiler can offer. The minimum choice is to compile with `-O3`, although some other compilers might offer further granularity. Also consider using `-ffast-math` which enables aggressive floating-point optimisation.

Second, we create code that is tailored towards our particular machinery. The correct setting for this depends once again on your compiler. With LLVM and GNU, you can use flags starting with `-mXXXX` to specify machinery-specific optimisations, i.e. you can exactly say for which processor generation, instruction set, ... you want for the compiler to create binary code. If your compile node is of the same type as your test node, then simply use `-xhost` and the compiler will pick the highest instruction set available. The flag is called `-mhost` with GNU.

Work in progress/todo A question which was raised at RSECon25: when applying aggressive optimisation, do we include validation checks on the code? I.e., to ensure that the validity of the code still passess when potentially using approximations in the higher level optimisation flags.

Intel compiler

With the Intel compiler, you sometimes can pick between the generic LLVM (community) versions of a feature and one optimised by Intel itself (with the latter obviously performing better on bespoke Intel architecture). For exam-

ple, `-fopenmp` enables the general OpenMP implementation whereas `-fiopenmp` gives you an in-house version of the same.

And while `-xhost` uses Intel-specific optimisations, `-mhost` will use LLVM's optimisation stack, i.e. Intel has replaced some optimisation steps with bespoke variants, which are enabled if and only if you go down the `-xhost` route. Be aware that Intel's optimisation passes typically will be disabled if you use the Intel toolchain on other vendor's hardware, i.e. you'll get unoptimised code.

Checklist

1. Your benchmark is compiled with the highest optimisation level (`-O3` plus a hardware-specific instruction set such as picked by `-xhost`).

MAQAO

If you run MAQAO over your code, the tool will report back to you what optimisations you might have missed out throughout the compilation. The top-level rubric *Global* provides you with information on the optimisation chosen, but it also makes recommendations which options might lead to better runtime.

The nice thing about MAQAO is that the tool can handle situations where you translate different translation units with different options. Also, the recommendation aspect goes beyond pure reporting and is helpful.

How it works

- ⊞ Use the most aggressive compiler optimisation that preserve the code semantics (aka still return valid results) and nevertheless are tailored towards your assessment machinery.
- ⊞ Check what optimisations the users' build system uses by default. Often, this is a low-hanging fruit.

How it does not work

- ⊟ Use a low optimisation level and machine-generic optimisations and then draw conclusions on code behaviour on a particular machine.

Chapter 8

Node-level (intra-node) analysis

Once we hit this section, we know that we make no particular good use of our cores. This is evidenced by the fact that our intra-node scaling is not good (cmp. Section 5.6), i.e. we measure what we get on one core and then extrapolate what our given number of cores should yield. But they don't: The speedup as introduced from (2.1) is poor as we increase the core count. We leave a lot of potential compute capability unused.

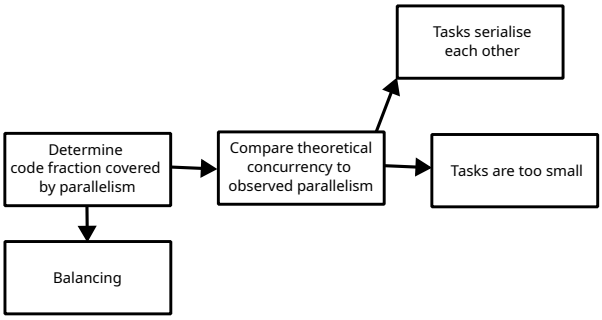


Figure 8.1: Decision graph for inter-node analysis.

Our deep dive starts from an analysis if there is enough concurrency in principle to exploit all cores. If there are not enough concurrent operations, further analysis is not required. The code can be returned immediately to the developers. Otherwise, we take it from there (Figure 8.1).

Preparation

Inter-node analysis is all about how efficiently we use multiple threads. Therefore, it focuses predominantly on relative quantities: How well does the code perform if we add another thread? However, we cannot study 128+ measurements in one go—even though modern performance analysis tools have quite powerful capabilities to compare different setups. We need to focus on few setups that yield interesting setup.

Assessment crime 15 *People study a whole-node run right from the start without a scalability analysis beforehand.*

It makes sense to run a strong scaling test and plot the efficiency. Once completed, we can flag those areas where the gradient suddenly changes or where the efficiency drops below 80% or 60% respectively. These are the setups (turning points) on which we should focus most of the time. Do not try to study any other setup, as you'll most likely be swamped with data that is not relevant or difficult to interpret!

How it works

- ⊞ Run a strong scaling test and plot the efficiency. Narrow down the number of setups of interest, i.e. identify regions where the efficiency apruptly changes or falls under a threshold.

How it does not work

- ⊟ Start with a full-node run right from the start and hope that you spot interesting trends in the data.

8.1 Coverage

Hypothesis 1 *If only a fraction of the code is parallelised and if this fraction is too small, this becomes the reason for poor (strong) scaling.*

To follow up on our working hypothesis, we analyse how much of a code is covered by parallelism, i.e. could in theory run in parallel. Our strategy to follow-up on the hypothesis that there might not be enough concurrency in a code covers studies along different dimensions. On the one hand, we can use a standard strong scaling model and fit it to measurement. If it fits, it yields an abstract metric of how much of the code is serial. We can then make follow-up decisions based upon this outcome. If it doesn't fit, this also provides us with intelligence. Parallel to that, we can employ tools and measure the observed parallelism (which is not the same as the theoretical concurrency). Again, this provides us with insight on the degree of code that is covered by parallel constructs and should spread out over cores.

8.1.1 A simple performance law

One of the simplest off-the-shelf models for parallel code is Amdahl's law [3]. It assumes that the runtime of a code is given by

$$t(p_{\text{threads}}) = t(1)\left(f_{\text{intra}}/p_{\text{threads}} + (1 - f_{\text{intra}})\right). \tag{8.1}$$

from which it argus on the obtained speedup or efficiency, respectively

$$
\begin{aligned}
S(p_{\text{threads}}) &= \frac{t(1)}{t(p_{\text{threads}})} \\
&= \frac{t(1)}{t(1)\,(f_{\text{intra}}/p_{\text{threads}} + (1 - f_{\text{intra}}))} \\
&= (f_{\text{intra}}/p_{\text{threads}} + (1 - f_{\text{intra}}))^{-1} \qquad (8.2)
\end{aligned}
$$

In this model, we assume that the time $t(p_{\text{threads}})$ for a code to finish the calculation of one fixed setup on p_{threads} threads depends only only on one variable that is specific to our code: There is a part (fraction) $0 \leq f_{\text{intra}} \leq 1$ of the code that benefits as we add more and more hardware threads. It scales perfectly. The other code fraction $1 - f_{\text{intra}}$ however runs serially and hence does not benefit from additional cores. If we want to know how long a code fitting onto one thread would need on p_{threads} threads, we take the fraction of time that scales and divide it by the number of threads. After that, we add the serial part, which stays invariant.

On top of this vanilla model, we have already implicitly used an additional penalty variable $0 < C_{\text{intra}} \leq 1$ in Section 5.5. It quantifies how much slower the code runs as we enable our multithreading implementation. It quantifies the sole parallelisation overhead. Adding this calibration into the model arising from (8.2) is straightforward.

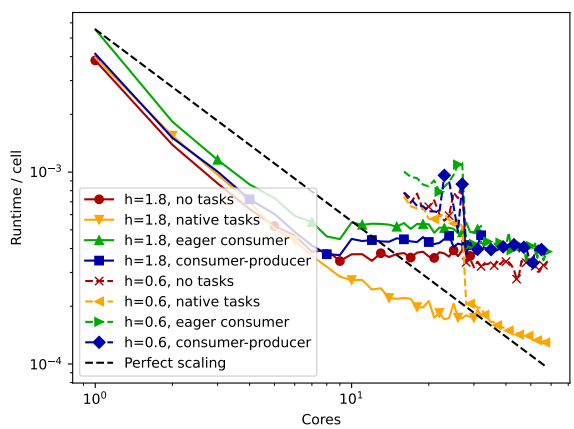


Figure 8.2: Scaling plots for different solvers for two different mesh sizes h.

Assessment crime 16 *Amdahl's law is used to derive statements on strong scalability using a handful of sample points without sanity checks if the model accurately describes the strong scaling behaviour overall.*

Calibration With the performance model in place, we can measure the actual performance and reconstruct f_{intra}. We can calibrate the equation to observed data (Figure 8.2).

There are, broadly speaking, three different outcomes:

1. The model fits, and $f_{\text{intra}} \ll 0.2$.

2. The model fits, but $f_{\text{intra}} \geq 0.2$.

3. The model does not fits.

In the example in Figure 8.2, the yellow line obeyes Amdahl's law albeit with a rather large f_{intra} value. The other plots "suddenly" plateau and hence do not fit the simplistic performance law.

Contextualisation Intra-node analysis' ultimate goal is to assess and provide guidance on a code's behaviour on a given system, i.e. for a given core count. In that sense, it seems weird to rely on an upscaling model. However, the upscaling model provides us with important insight that we can use to narrow down the search for performance flaws.

8.1.2 Code coverage

As follow-up to the symptomatic studies, we can dive into the code and analyse how much of it is actually covered by parallel constructs. Studying the code to see how many lines are covered by an OpenMP pragma for example is an option to gather intelligence. Far more effective is using a tool that measures how much code is run in parallel. Using a tool also has the advantage that it provides weights to the lines: A line that is executed only once might not have to run in parallel, while instructions that are executed thousands of time make the runtime potentially deteriorate. Knowing interesting sample points, using a tool becomes feasible, as we do not have to study all potential core counts.

VTune

VTune can provide you coverage analysis through its Flame Graphs:

In the example above, the OpenMP core routines are clearly visible and provide information that all parts of this code are covered properly by OpenMP parallel regions. In this example, the concurrency level is, in principle, high.

8.1.3 Extensions of Amdahl's law

There are a couple of issues with this model which make it often struggle to represent real-world data:

- The formula assumes that we have either sections that are parallel or sections that are sequential. In real codes, we often have sections that use some tasks or small loops. These do not scale with p_{threads} which the model will fail to capture.
- If a code is heavy on semaphores and critical sections, it will have to serialise some parts of the code. They cannot run in-parallel. This will lead to a blurred parallelism—maybe even non-deterministic.
- Every parallelisation induces overhead to manage threads, realise dependencies or synchronise parts of the code. This overhead grows as the number of threads grows.

Due to these effects, many codes will yield data where the runtime starts to raise again once we exceed a certain core count (yellow dots in Figure 5.1). This could be modelled by an additional penalty term $C_{\text{penalty}} \cdot p_{\text{threads}}$ added to (8.1). In other cases, we would have to add a noise term depending upon p_{threads}. Anyway, we note that a measured f_{intra} is an effective parallel fraction, i.e. something we see. It does not mean that the code really is a simple sequence of loops which all scale brilliantly, while the parts in-between can be accurately quantified by $1 - f_{\text{intra}}$.

Despite all of its shortcomings, we can use the model and actually turn the argument around: If we can use Amdahl's law (8.2) to model a code accurately and if the resulting f_{intra} is sufficiently big, then we are on track to deliver a code which is of high intra-node efficiency. Otherwise, we will continue to dive into the code anyway.

8.1.4 Evaluation

- ⊠ If the strong scaling model fits and $f_{\text{intra}} \ll 0.2$, the code should in principle scale. It is not likely that it has a strong scaling issue. However, it might be appropriate to play around with different rank per node settings, i.e. to increase the number of MPI ranks mapped onto one node such that each rank has effectively fewer cores to employ. This way, the serial fraction f_{intra} would affect the efficiency less significantly.
- ⊠ The model fits, but $f_{\text{intra}} \geq 0.2$ despite the fact that the concurrency analysis suggests that all code parts are parallelised. Even though the code is parallelised, it effectively serialises major parts of the code. In many cases, this can result from ill-balancing or excessive synchronisation.

  ⇒ Look into the balancing (Section 8.2). And imbalanced load distribution leads to an increased pseudo-serial code fraction.
- ⊠ The model fits, but $f_{\text{intra}} \geq 0.2$ and major code parts are not captured by the parallel regions. Significant parts of the code are not parallelised.

⊠ The model fits up to a certain core count, but then plateaus.

⇒ Ensure that the code's concurrency level is sufficiently high, i.e. that there should, in theory, be enough parallel work (Section 8.3).

How it works

⊞ Use Amdahl's formula to find out if your code follows the simplistic scaling law qualitatively and quantitatively.
⊞ Employ a tool to validate sufficient code coverage.

How it does not work

⊟ Run a few arbitrary experiments, determine f_{intra} through regression, and draw general conclusions about your code.

8.2 Balancing

Hypothesis 2 *The code is split up into a sufficiently large chunk of concurrent sections, but these sections are of massively different size. It is this imbalance that harms the performance.*

If work is imbalanced, some cores will idle at one point. If we do not exploit all cores all the time, we cannot expect to squeeze out all of the performance of the node.

8.2.1 Model

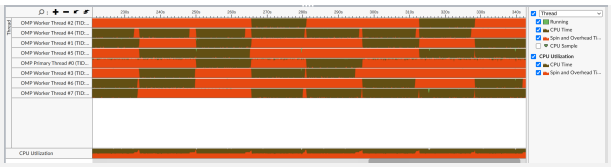
Performance metric (rule of thumb) 6 *In line with all other metrics, we consider a code phase or loop reasonably balanced, if it is able to exploit around 80% of the available compute resources at any point in time. If we underrun this threshold, the code is not super balanced.*

8.2.2 Data collection

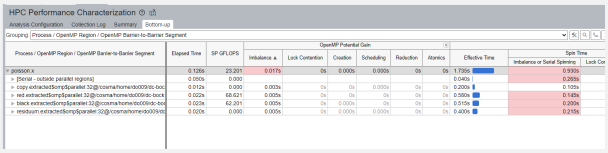
Most profilers have quite an extensive support to identify ill-balancing. After all, this is the most obvious, ultimative, original sin of non-scaling.

VTune

The hotspot and threading analysis of VTune both offer you a bottom-up view, where imbalances are clearly visible:



The idle threads here don't really idle: They continue to run even if they have nothing to do, but spin all the time, i.e. wait busily for new work to drop in. We can dig deeper with the *HPC Performance Characterization* analysis, where we see imbalances broken down over individual functions or loops, respectively. The mode is particularly useful for loop-/data-parallelism. For code that relies a lot on tasks, it is not straightforward to localise imbalances, and we might have to do some manual analysis work.



8.2.3 Evaluation

- ⊠ The work is perfectly balanced, but the performance remains poor.
  - ⇒ Study the average task size and investigate whether these tasks can be split up further (Section **??**).
- ⊠ The work is not perfectly balanced.
  - ⇒ Natural performance optimisations are switching to smaller grain sizes (iteration space chunks) or to enable dynamic scheduling if the individual chunks have largely varying runtimes. However, these are already code tweaks, i.e. beyond the scope of this handbook.

How it works

- ⊞ Track how long the cores actually compute and compare it to the time where they do not deliver science.
- ⊞ If there are severe imbalances, continue to work through hypotheses to find out where the imbalances come from and reduce the granularity such that there are enough pieces of work.

How it does not work

- ⊟ Stop the analysis once the work is perfectly balanced. Performance might still suffer from overheads.

8.3 Concurrency level vs. observed parallelisam

Hypothesis 3 *There is not enough work, i.e. the concurrency level is not high enough.*

Finding out how much work there should, in theory, be in a code is not trivial. We roughly distinguish two ways to conduct these studies: a symptomatic approach and a tool-based one.

8.3.1 Tool-based analysis

Work in progress/todo Not clear what the right tool is to observe this. OpenMP's tool interface might have an option.

8.3.2 Symptom analysis

Arguing about a lack of concurrency can be relatively simple or utterly complex. A simple occupation analysis might demonstrate that some code phases just do not yield enough work. Poor balancing (cf. Section 8.2) for example materialises immediately in low concurrency.

Things are more difficult for task-based parallelisation approaches. If there are not enough tasks, the task granularity is not high enough. Here, insufficient concurrency

Work in progress/todo Not clear how this is best done. I had a code with thousands of tasks, but the dependencies were so strict that the number of ready tasks per step was actually way too small. Not sure how to see this. For loops with too small grain sizes, things should be simple.

8.3.3 Evaluation

- ⊠ There is simply not enough work and some cores idle.
  - ⇒ Look into the balancing (Section 8.2) if a code relies on classic data parallelism (parallel loops).
  - ⇒ Study the average task size and investigate whether these tasks can be split up further (Section **??**).
- ⊠ The concurrency level unfolds hierarchically—e.g., tasks spawn further tasks or loops host other parallel loops—but the parallelism does not increase.
  - ⇒ Ensure that the tasks do not kick off with an extensive serial prologue.
  - ⇒ Ensure that hierarchical parallelism is properly supported (Section 8.12).
- ⊠ There are a lot of tasks, but the effective CPU usage is low.
  - ⇒ Study the task dependencies and evaluate the actual number of ready tasks at each time point (Section **??**).

Work in progress/todo We have seen such a case but only found it through intuition, i.e. not in a systematic way. Not clear how this could have been seen.

⊠ The concurrency level is high all the time. Nevertheless, the performance remains poor.

- ⇒ The parallel loop segments might be too small (Section 8.5).
- ⇒ The tasks might be too small in a task-based parallelisation (Section 8.4).
- ⇒ Tasks might lock each other out all the time, but only for a very brief duration (e.g., short call inside a loop). So it looks as if things ran in parallel, but they do not really do (Section 8.13).

How it works

⊞ Study if there is always enough work in theory available at any point.
⊞ Either employ a tasking tool or argue through observations.

How it does not work

⊟ Try to explain poor behaviour without having ensured that there is enough concurrency, i.e. work available in theory.

8.4 Tasks are too small

If tasks (or parallel sections) are too small, we pay an excessive overhead. The concurrency level might be brilliant, but that does not help us if we have too much task creation and task clean-up overhead.

Hypothesis 4 *There are enough tasks and they are not excessively synchronising each other, i.e. there are not that many dependencies, but they are just too small and hence create too much administration overhead.*

There is not really a definition of a "too small task". However, we can use rule of thumbs from parallel loop segments and consult popular developer platforms and read about the recommendation that a reasonable task should at least run for tens of microseconds. The extreme case of tiny tasks are empty tasks which some codes use to model dependencies, i.e. to synchronise sets of tasks in a task graph. If we have too many of those guys, the fine granular decomposition of a problem into a task graph becomes a performance problem in its own right.

8.4.1 Data collection

Different to excessive synchronisation (Section 8.8), spotting too small tasks is often challenging. Either, we do not see locks or excessive spinning at all, or we see these but within the task runtime itself. After all, small tasks lead to a low overhead from a user code's point of view, as the overhead is typically attributed to the task runtime.

A plain profile is difficult to assess as well, as very short task functions do not appear high up on the list either. They might have a very high call number, but the combination low runtime plus high call number also implies that they are often hard to distinguish from sole helper functions.

Due to the fact that this flaw is not trivial to spot, it does not come as a surprise that many task-based codes bring along their own task tracing. The present manuscript cannot and should not cover these bespoke solutions, but rather advocates for a symptomatic analysis or, if possible, measurements through a runtime's tool interface.

Symptom analysis with OpenMP

For systems employing OpenMP, very small tasks materialise in a lot of time spent in `__kmp_fork_barrier`. When we open up the trace, we see that this function is called by `__kmp_launch_thread`. So all the overhead arises from `libiomp5.so` or similar.

VTune

In VTune's hotspot analysis, you see in the summary a very high overhead time. Indeed, studies of the timeline confirm that we see that all the time is burn in spin and overhead.

In VTune's Threading analysis as well as the HPC Performance Charaterization, we are informed on the summary page that the Effective CPU (or physical core) Utilization is low. Low here implies less than 80%. Unfortunately, all the tools claim that the OpenMP region with the biggest potential gain is something similar to `unknown$omp$parallel`, and if we zoom into the area of interest, the threading analysis quickly tells use that there is no data to display.

Work in progress/todo I haven't yet found a way to find out which tasks are too cheap.

8.4.2 Evaluation

- ⊠ There are too many tiny tasks modelled as real tasks.
  - ⇒ Inline tasks, i.e. call the tasks as functions rather than spawning them as real tasks.
  - ⇒ Randomise task creation and therefore throttle the task production.

In OpenMP, ensuring that logical tasks are not mapped onto physical tasks is very simple. You can add a `if (xxx)` annotation to the pragma that switches it on or off depending on a boolean predicate.

8.5 Loop segments are too small

8.6 Tasks are too big

8.7 NUMA effects

8.8 Synchronisation

8.9 Data dependencies

8.10 Redundant (administrative) work

8.11 Homogeneous task duration

8.12 Hierarchical decomposition

8.13 Excessively fine-granular task synchronisation

Hypothesis 5 *Tasks block each other for very short periods. It looks like they ran in parallel, but they are actually sequentialising each other.*

This problem arises for example if there are lots of tasks hosting a for loop which in return has a very small critical section: The tasks lock each other out and effectively serialise each other while the execution threads are interwoven into each other. A cartoon might help

8.13.1 Diagnosis

If there are many critical sections following each other, this manifests in a lot of spin time in the multithreading back-end. Performance analysis and tuning instructions recommend to increase the task granularity or the scope of the data synchronisation (cited from VTune).

VTune

The code reports significant overhead time.

Part IV

Resources

Chapter 9

Tools

A lot of our methodology and research questions (cmp. introductory remarks in Chapter **??**) are inspired by VI-HPS and the tools represented within the consortium. The visualisation tool Cube [4] for example shapes the discussion on localisation. Consequently, all the tools that can feed into Cube are natural candidates to answer our most of the questions asked throughout the subsequent in-depth studies. However, there are a lot of alternatives out there. Some of them work for all three flavours, some of them work particularly well for one or two.

While we discuss some tools in this chapter, this list is not comprehensive and might over-simplify certain aspects. We might claim that tools are particular good for one thing while expert users and their developers know that they can also be used for others.

9.1 GNU profiler (gprof)

Every Linux system is shipped with the GNU profiler (`gprof`) which is therefore a convenient starting point for every a performance analysis. `gprof` is based upon instrumentation: You ask the C/C++ or Fortran compiler to instrument (Section 2.3) your code by passing in `-p` to both the compiler and the linker step. After that, the code's execution will monitor the behaviour and dump the observation into a file `gmon.out`. By typing in `gprof ./code`, you tell the profiler to take the executable's debug information (remember to translate with `-g`) and bring it together with the profile to assemble some meaningful stats.

The outcome consists of two major sections: The code first enlists the hotspots, i.e. the most expensive functions, and then it provides a call graph. All the output is purely text-based by default. There is no need for a fancy GUI or some X-forwarding.

Overview

- ⊞ Available everywhere.
- ⊞ All text-based.

- ⊟ You have to recompile.

- ⊟ Limited filter features to reduce the measurement overhead.
- ⊟ Cannot distinguish between different threads, and is not so straightforward to use with MPI.
- ⊟ Focuses on timings and call counts only.

9.2 Python's on-board profiling

9.3 APS

Intel Performance Application Snapshot (APS) is an analysis tool which generates high-level overview/summary pages of how a code performs. The usage is close to trivial:

```
mpirun aps ./code args
#aps -report result folder
aps-report result folder
```

which should create a plain HTML file to study. If you code does not use MPI, skip the `mpirun`.

The tool works best on Intel machinery, as it can read out the machine's performance counter. These counters are not available (that way) on other vendors' machines. In this case, you can restrict APS to focus on certain modi through `aps --collection-mode=mpi,openmp`.

Intel Performance Application Snapshot (APS) provides a global overview of the code's behaviour.

Further to that, you might need `#SBATCH --disable-perfparanoid`

Overview

- ⊞ Simple to use with support for MPI, share memory parallelisation, I/O and vectorisation features.
- ⊞ Yields simple HTML pages which are easy to share and archive.

- ⊟ Provides the full range of information only on Intel CPUs.
- ⊟ Limited filter features to reduce the measurement overhead.
- ⊟ Cannot distinguish between different threads, and is not so straightforward to use with MPI.
- ⊟ Focuses on timings and call counts only.

9.4 Intel VTune

Intel gives its VTune profiler away free of charge. The tool provides a fancy GUI that you can use to study your code interactively, but there's a button at the start screen that tells you how to invoke the profiler on the command line. Through this, you can run the profiler on a compute node without the GUI, and then copy the results back to your

workstation for a posteriori assessment. Once available (on your supercomputer you'll have to load the VTune module, while a `source /myinteldir/setvar.sh` sets all variables on a local installation), a simple `vtune-gui` brings up the tools.

VTune's various data collection modes are based upon sampling, and there's the opportunity to dive very deeply into the machine's behaviour. As the tool displays the code analysis over time and cores, you can zoom into certain runtime phases and cores manually. The delivered data is more detailed on Intel machines, but the tool also is very useful on AMD. Some of the more modi require certain system priviliges.

Overview

- ⊞ Works also on non-Intel machinery.
- ⊞ Very detailed node- and core-level data including performance counter data.
- ⊞ Can trace MPI.

- ⊟ Some in-depth features allow special system rights that normal users typically don't have.
- ⊟ Very easy to get lost in massive amounts of data.

9.5 Score-P

9.6 Linaro Map

9.7 MAQAO

MAQAO has the high-level report but it has this ambition in its DNA to guide the tuning and development. In that sense, it goes beyond mere analysis and has a slightly different focus than a lot of the other analysis tools. While MAQAO gets more support for multithreaded and MPI runs with every new generation, its core expertise is the analysis on the core level: Which loops consume most runtime, vectorise poorly or are not balanced reasonably? That's the type of prime application area of MAQAO.

9.8 Intel Amplifier

Part V

Case Studies

Chapter 10

High-level assessments

Work in progress/todo In this chapter we aim to include example high-level performance assessments applied to a handful of code examples. These high-level assessments are reports which are generated from a Jupyter notebook (currently in development).

Part VI

Appendix

Appendix A

Acknowledgements

A.1 Funding councils and supporting initiatives

This work was supported by the Science and Technology Facilities Council [grant number UKRI/ST/B000293/1]. This work also was supported by the Engineering and Physical Sciences Research Council [grant number UKRI1801]. The underlying projects HAI-End and SHAREing (Skills Hub for Accelerated Research Environments Inspiring the Next Generation) is part of the cross-UKRI Digital Research Infrastructure initiative.

A.2 People

This manuscript is the result of many meetings, workshops and conversations, and many people chipped in ideas and remarks that eventually helped us to improve the text. Special thanks are due to

- Brian Wylie
- Ben Clark who knocked down the first prototype of the accompanying Jupyter notebook in only one week of his school placement.
- Eva Fernandez Amez who created all the upskilling videos that we created whenever we spotted topics at the workshops which deserve a brief revision.

A.3 Partners

We thank the companies AMD, Intel and Linaro for their continuing support of these activities by sending delegates to our workshops and contributing to our document.

Durham University is a proud partner of VI-HPS (https://www.vi-hps.org/) and appreciates the input of our colleagues and friends of the research consortium. Without their input, none of this would have been possible.

Appendix B

Submission checklist

This chapter hosts a set of checkpoints that groups might introduce *before* they start to conduct assessments and analyses. They form kind of a common sense contract between the code owner and the person conducting a review and ensure that some typical hiccoughs and problems are avoided. As it has this legal flair, we call the parties involved the code *owner* vs. the code *analyst*, even though these roles in practice might be blurred and taken up by code developers themselves. The checklists complement the aspects and checklists discussed in Chapter 4. While Chapter 4 is written from an analysis/assessment point of view and hence targets predominantly the analyst, the checklists here are more owner-focused and notably highlight the interaction between analysts and owners.

B.1 Code submission

Before we accept a piece of code for the assessment and analysis, it is important to identify what we expect from the owner submitting the code. This covers first of all the artefact itself:

Checklist (Code)

1. There is a central archive with all files required to run the code. The archive is self-contained.
2. This archive is frozen, i.e. no one changes the object of an assessment while the assessment is going on.
3. There is a well-defined, characteristic run of the code which the analyst can use to study performance of the software. This benchmark should not take 'too long'; for further details please see the SHAREing benchmark guidelines.

The second point is obviously once written down, but deserves to be highlighted: If an object of study changes as we try to assess it, it is virtually impossible to get the

big picture right. Obviously, continuous benchmarking and performance analysis can and should be part of professional scientific software development. However, if we try to conduct an objective assessment, we have to work with a fixed object of study. That means, we should freeze the development for the time being or at least work on a stable (release) branch. For example, the owner can create a *tag* within the repository. Tags are often used to annotate specific code release versions, but here we can also tag the repository at a specific point for assessment. This can be achieved simply via the `git tag` command. If a code does not (yet) support all dimensions of an assessment (cmp. Chapter 5) or some dimensions are not yet stable, this is the time to flag this, so we can exclude them from the analysis.

Submitting a characteristic run means that the owner describes how we can start such a run on the reference system ourself. This is the litmus test: If we cannot (even) start a baseline run, then the code/submission is obviously not yet in a shape to be assessed and we have to stop immediately.

The attribute "characteristic" requires the owner to doublecheck if the setting handed over makes sense. Both parties want the outcomes of the benchmarking and analysis to be meaningful. We assume that the owner wants to learn something about the code from an objective analyst, while the analyst wants to know right from the start that their work is relevant. We want to avoid a situation where we are told afterwards that the analysis' results are meaningless because "in the real world, we would use a completely different setup". This has happened to us before.

Further to the mere artefact submission, we also require more detailed information from the owner. We begin by outlining the required documentation that we expect from the owner:

Checklist (Documentation)

1. Software dependencies, e.g., libraries such as FFTW, LINPACK, etc. preferably with versions if known.
2. Compilers, preferably with version if known, along with the typical compile flags.
3. Parallel libraries/models, e.g., MPI, hybrid parallelism with MPI+X, CUDA, OpenMP, etc.
4. Detailed build instructions.
5. Run instructions, i.e., the necessary run commands including any relevant details of the command line interface for passing arguments to the executable.

This required documentation we see as necessary for an analyst to start by cloning the repository, and almost without thinking, produce a correct executable for the "characteristic" code run. Though we treat the code as a black box, the parallel model is requested to guide the analyst to which aspects of a software's performance can be analysed, i.e., clearly a code which supports MPI only cannot have its GPU performance analysed.

With this information an analyst should be able to build the executable and so next

we request typical resource allocations for the software such that the analyst can begin to define job scripts and plan out the assessment. The requested resource allocation information is listed as:

Checklist (Resources)

1. The owner should, if known, state some example runtimes of the characteristic benchmark along with the hardware examples and configurations that were used.
2. The owner should submit example configurations of the parallel resources used. For example, if CUDA+MPI is used then how many ranks are assigned per GPU? If using OpenMP+MPI then how many threads are typically assigned per rank.

This information is to direct the analyst towards 'typical' job and resource allocations. For example, some hybrid parallel MPI+OpenMP codes use quite specific allocations of ranks per node and threads per rank, and it is not feasible to explore all combinations. Hence, requesting an example job setup, preferrably in the form a job script, can direct the analyst and make the performance report more relevant to the owner. Furthermore, knowledge of typical runtimes along with job setups can be useful for the analyst to sanity check results and support that the benchmark configuration is in keeping with the guidelines touched on in Section 4.1.

Having built the executable and designed the relavent job scripts, we finally request information on benchmark configuration:

Checklist (Configuration)

1. The owner should describe how to vary the benchmark, i.e., which flags can be varied including typical value ranges for these parameters.
2. The owner should supply details of how the benchmark scales with the input parameters.
3. The owner should describe how, if possible, the I/O of the benchmark can be switched off or on.

The questions on configuration — at least for the high-level assessment — are mainly for a weak scaling analysis, because this is likely the only point in which the analyst is straying away from the example benchmark. Therefore, the analyst needs to understand how to scale the benchmark both from an interface standpoint of how do we vary the input parameters, but also how does the algorithm scale. Put simply: how do we vary the input parameters to, e.g., double the problem size?

The final question on I/O is that when not studying I/O performance, it can be beneficial for the analyst to be able to switch this off and avoid I/O skewing performance measurements.

All of the requested information above is to aid the analyst to be able to perform our high-level assessment as just "turning the handle". However, technical documentation

of a code is not all that is required for a performance assessment service; professional conduct and managing expectations.

B.2 Code of conduct

Above we discussed the importance of defining characteristic benchmarks to avoid (from experience) performance reports being disregarded as not indicative of "real world" runs. Along the lines of "this has happened to us before", we here write down some simple principles how to behave regarding the analysis:

Checklist

1. The owner is available to the analyst for questions around the code while the assessment project is going on, and try to answer questions re code usage and setups quickly and constructively.
2. The owner will refrain from challenging the assessment methodology.
3. The owner commits to acknowledge all.

The first item can be subject of debate.

Work in progress/todo Greater context needs to be added here.

B.3 Outcomes

It is important to sort out what happens with any outcomes before the assessment starts. The two items below can be subject of debate, but they align with principles of open, fair and reproducible science:

Checklist

1. The owner grants the analyst the right to publish their review.
2. The analyst commits to an objective assessments which refrains from "blaming" a code for certain behaviour.

This sounds defensive and it is defensive, but we have observed numerous situations where developers reacted annoyed up to angry when they read assessments. Scientists tend to develop emotional attachment to their codes, and are not amused if an independent analysis highlights shortcomings. It is important to acknowledge that this is exactly the purpose of an analysis. It is not there to confirm how exceptionally great some piece of code is already, but should inform future development.

Appendix C

Choice of Metrics

This appendix details some of the reasoning behind our decisions on which metrics to use for each assessment rubric.

C.1 GPU Metric: Occupancy or FLOPS?

In initial work on the methodology, it was thought that the average warp occupancy over program runtime might be used as a workable metric to evaluate the GPU usage of a code. However, as we developed the methodology and began producing example high-level analyses, we found cause to question whether this was in fact the right choice. The alternative proposed at this point was to use the same metric as for the CPU rubric, floating-point operations per second, a measurement of rate-of-work-done.

First, rate of work is more intuitive to code owners than hardware occupancy, and reaching high rate of work compared to hardware theoretical max is correlated with occupancy in hardware utilisation terms. Second, rate of work is closer to what we actually care about in performance assessment. Third, and possibly most importantly, occupancy is not always correlated with a higher rate-of-work. For a basic example code, the block size was set to 256 (eight times the size of a warp), and the number of blocks was set to 400 (five times the number of streaming multiprocessors in the Tesla V100 GPU the code was tested on). With this configuration, the occupancy was observed to be 62.4%, and the rate of work about 15.6 TFLOP/s. This was the highest rate of work observed in all configurations tested (although only three configurations are expanded on here). Increasing the number of blocks to 1600 (twenty times the number of streaming multiprocessors), the occupancy was observed at about 92%, but the rate of work was lower at about 14.6 TFLOP/s. Additionally, due to the architectural specifics of GPUs, increasing the number of blocks to 401 increased occupancy to 62.6%, and decreased rate of work to 13.1 TFLOP/s.

Appendix D

Communication with code owner

There are many times throughout an assessment where direct communication may be required. For now we list the area of 'Existing Optimisations', which is a tricky scenario to properly include in an assessment.

D.1 Existing optimisations

Reader's guide

Playing around with (user-driven) optimisations can become tricky and might require the assessor to communicate with developers (if you are not the same person). For an initial assessment, it might be very reasonable to skip this section.

It is rare that we work with a code that hasn't gone through a series of optimisation steps already. Most developers think at one point about how to make a realisation fast. However, optimisation steps can backfire:

First, many code developers have no systematic training in HPC and/or do not apply rigorous performance analysis. Consequently, their realisation does not tackle hotspots. In the worst case, the resulting optimisation steps make the code more complicated than it actually has to be.

Second, many scientific codes are old. They contain legacy code parts. If these code parts had been optimised, the optimisation steps might not be valid for today's computer architectures anymore. For example, compute time used to be expensive and performance engineers hence used to introduce precomputed data tables in the past. Today, memory bandwidth is likely the most precious resource. Having extensive lookup tables can actually impose additional stress on the memory interconnect, and a recomputation of values might be advantageous.

Finally, we have to be frank with our community: A lot of people who consider

themselves to be outstanding computational scientists actually have a very limited understanding of computer architecture. That's not a problem per se, as long as people are aware of their lack of knowledge. It is problematic when we see developers introducing or pushing for "optimisations" which actually are not supported by data or insight, but rather individual "genius" and "expert knowledge".

Assessment crime 17 *We work with a code that the developers consider to be "highly optimised". Due to these optimisations, we start from a code base that is actually overly complicated (thus hiding the "real" performance flaws) and might actual underperform* due to *these optimisation steps.*

Many scientific codes have not been co-designed as HPC codes. They have been written first and foremost with functional requirements ("the science") in mind. The HPC came later. This is reflected in the code design: Optimisations are realised as add-ons wrapping around the core science (yet sometimes, unfortunately, also penetrating it). Therefore, we always have to keep in mind that performance has been seen as a second-class citizen in most development projects. By the time a project has reached a certain maturity and people suffer from performance flaws, they will ask for a performance assessment (and then want to be told that they have already done the heavy lifting or that their science is so challenging and completely different to everything else that it is very hard to get good performance). They won't ask for it prior to that.

If possible, it is helpful to disable all of these historic optimisations. We can then start with the performance assessment on a green field and re-inject the optimisations one by one as we go along. In an ideal world, they should make the code faster, but this has to be validated!

Bibliography

[1] Kent. Beck. *Extreme programming explained : embrace change*. Addison-Wesley, Reading, Mass, 1999.

[2] John L. Gustafson. Reevaluating amdahl's law. *Commun. ACM*, 31(5):532–533, May 1988.

[3] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, AFIPS '67 (Spring), page 483–485, New York, NY, USA, 1967. Association for Computing Machinery.

[4] Markus Geimer, Björn Kuhlmann, Farzona Pulatova, Felix Wolf, and Brian JN Wylie. Scalable collation and presentation of call-path profile data with cube. In *PARCO*, pages 645–652, 2007.